

01. Clustering Algorithms

Algorithms implemented: K-Means, Modified K-Means (K-Means++ / Bisecting), Hierarchical (Ward), Fuzzy C-Means.

Dataset: Mall Customers (`data/mall_customers.csv`) - Annual Income vs Spending Score.

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, BisectingKMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (Linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Libraries successfully imported!

```
# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)
```

Dataset Shape: (200, 5)

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6

1. K-Means Clustering

Step-by-Step Algorithm (Lloyd's K-Means)

1. **Choose K** and initialize K centroids (here with K-Means++ for a strong start).
2. **Assignment step:** assign every sample to the nearest centroid (Euclidean distance).
3. **Update step:** move each centroid to the mean of the samples assigned to it.
4. **Repeat** steps 2-3 until assignments/centroids stop changing or the iteration limit is reached.
5. **Output:** final centroids, hard cluster labels, and the inertia (WCSS) used for model selection.

How K Is Selected in This Notebook

1. Fit K-Means for K = 2...10.
2. Record inertia (elbow curve) and silhouette score for each K.
3. Pick the K at the "elbow" / highest silhouette (K = 5 here) and refit the final model.

```
# ---- Selecting the optimal number of clusters K ----
# Try K = 2..10 and record two diagnostics:
# * Inertia (WCSS): total squared distance from points to their centroid
# * Silhouette score: cluster cohesion vs. separation
k_range = range(2, 11)
wcss = []
silhouette_scores = []

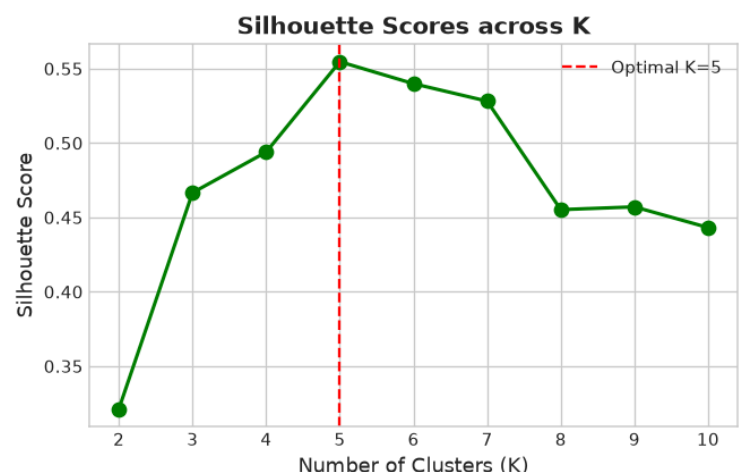
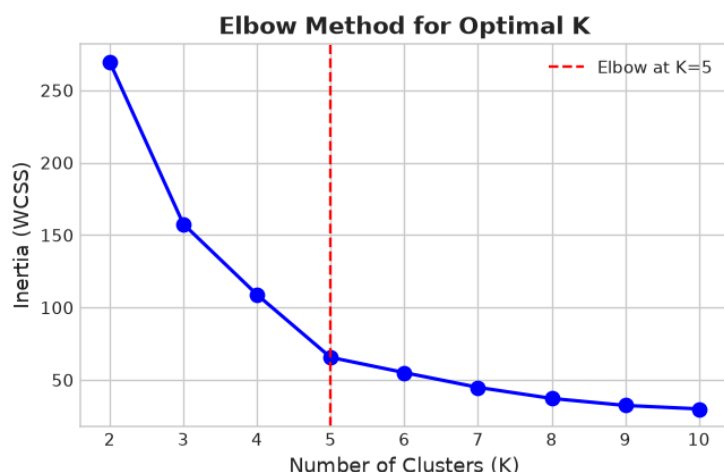
for k in k_range:
    # n_init=10 runs K-Means from 10 different seeds and keeps the best run
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    kmeans.fit(X_mall_scaled)
    wcss.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_mall_scaled, kmeans.labels_))

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# ---- Left: elbow curve (look for the "knee" where inertia stops falling quickly) ----
ax1.plot(k_range, wcss, 'bo-', linewidth=2, markersize=8)
ax1.set_title("Elbow Method for Optimal K", fontsize=14, fontweight='bold')
ax1.set_xlabel("Number of Clusters (K)", fontsize=12)
ax1.set_ylabel("Inertia (WCSS)", fontsize=12)
ax1.axvline(x=5, color='r', linestyle='--', label='Elbow at K=5')
ax1.legend()

# ---- Right: silhouette score for each K (higher is better) ----
ax2.plot(k_range, silhouette_scores, 'go-', linewidth=2, markersize=8)
ax2.set_title("Silhouette Scores across K", fontsize=14, fontweight='bold')
ax2.set_xlabel("Number of Clusters (K)", fontsize=12)
ax2.set_ylabel("Silhouette Score", fontsize=12)
ax2.axvline(x=5, color='r', linestyle='--', label='Optimal K=5')
ax2.legend()

plt.tight_layout()
plt.show()
```



```
# ---- Final K-Means model with the selected K = 5 ----
optimal_k = 5
kmeans_model = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20, random_state=42)
kmeans_labels = kmeans_model.fit_predict(X_mall_scaled) # hard cluster assignment per sample

print(f"K-Means Silhouette Score: {silhouette_score(X_mall_scaled, kmeans_labels):.4f}")
print(f"K-Means Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, kmeans_labels):.4f}")
```

K-Means Silhouette Score: 0.5547
K-Means Davies-Bouldin Index: 0.5722

2. Modified K-Means

Step-by-Step Algorithm (Modified K-Means Variants)

A. K-Means++ initialization

1. Pick the first centroid uniformly at random from the data.
2. For every point compute $D(x)^2$ = squared distance to the nearest chosen centroid.
3. Pick the next centroid randomly with probability proportional to $D(x)^2$.
4. Repeat steps 2-3 until K centroids are chosen, then run standard K-Means. *Benefit*: avoids poor random starts, converges faster and more consistently.

B. Bisecting K-Means (divisive)

1. Start with all samples in a single cluster.
2. Select the cluster with the largest inertia (or largest size).
3. Split it into two using 2-means (try multiple restarts, keep the split with the lowest SSE).
4. Repeat steps 2-3 until exactly K clusters are obtained. *Benefit*: more balanced, stable cluster sizes and less sensitivity to initialization.

```
# ---- Modified K-Means ----
# (a) K-Means++ initialization is already used in the standard model above.
# (b) Bisecting K-Means repeatedly splits the cluster with the largest inertia until K clusters.
bisecting_km = BisectingKMeans(n_clusters=optimal_k, random_state=42,
bisecting_strategy='biggest_inertia')
bisecting_labels = bisecting_km.fit_predict(X_mall_scaled)

print(f"Bisecting K-Means Silhouette Score: {silhouette_score(X_mall_scaled, bisecting_labels):.4f}")
print(f"Bisecting K-Means Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled,
bisecting_labels):.4f}")
```

```
Bisecting K-Means Silhouette Score: 0.4807
Bisecting K-Means Davies-Bouldin Index: 0.6788
```

3. Hierarchical Clustering (Agglomerative)

Step-by-Step Algorithm (Agglomerative / Ward)

1. Start with N singleton clusters (one per sample).
2. Compute pairwise distances between all clusters using Ward's criterion: $\Delta W(A,B) = (n_A n_B)/(n_A+n_B) \cdot \|\mu_A - \mu_B\|^2$.
3. Merge the two clusters with the smallest ΔW (least increase in within-cluster variance).
4. Update distances between the new cluster and all remaining clusters.
5. Repeat steps 3-4 until a single cluster remains.
6. Cut the dendrogram at the chosen height (or choose K) to obtain the final partition.

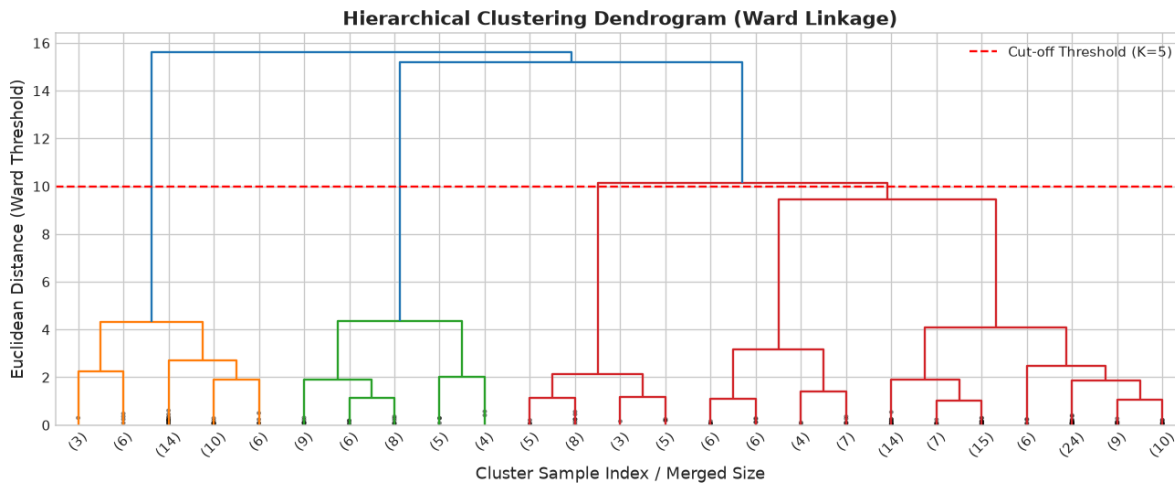
```
# ---- Build the Linkage matrix and draw the dendrogram ----
# 'ward' linkage merges the pair of clusters that increases within-cluster variance the least.
linkage_matrix = linkage(X_mall_scaled, method='ward')

# The dendrogram shows every merge; we truncate it to the last 25 merges for readability.
plt.figure(figsize=(12, 5))
dendrogram(
    linkage_matrix,
    truncate_mode='lastp',    # show only the last p merged clusters
    p=25,
    leaf_rotation=45,
    leaf_font_size=10,
```

```

show_contracted=True      # condensed leaves are shown as counts
)
plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)", fontsize=14, fontweight='bold')
plt.xlabel("Cluster Sample Index / Merged Size", fontsize=12)
plt.ylabel("Euclidean Distance (Ward Threshold)", fontsize=12)
plt.axhline(y=10.0, color='r', linestyle='--', label='Cut-off Threshold (K=5)') # horizontal cut =>
cluster count
plt.legend()
plt.tight_layout()
plt.show()

```



```

# ---- Fit the final Agglomerative model (Ward Linkage, K = 5) ----
agg_model = AgglomerativeClustering(n_clusters=optimal_k, metric='euclidean', linkage='ward')
agg_labels = agg_model.fit_predict(X_small_scaled)

print(f"Hierarchical Silhouette Score: {silhouette_score(X_small_scaled, agg_labels):.4f}")
print(f"Hierarchical Davies-Bouldin Index: {davies_bouldin_score(X_small_scaled, agg_labels):.4f}")

```

Hierarchical Silhouette Score: 0.5538
 Hierarchical Davies-Bouldin Index: 0.5779

4. Fuzzy C-Means (FCM)

Step-by-Step Algorithm (Fuzzy C-Means)

1. Choose K , fuzzifier $m > 1$ and tolerance ϵ ; initialize the membership matrix U randomly so each row sums to 1.
2. **Centroid update:** $v_k = \sum_i u_{ik}^m x_i / \sum_i u_{ik}^m$.
3. **Membership update:** $u_{ik} = 1 / \sum_j (||x_i - v_k|| / ||x_i - v_j||)^{2/(m-1)}$.
4. **Repeat** steps 2-3 until $\max |U_{\text{new}} - U_{\text{old}}| < \epsilon$ or the iteration limit is reached.
5. **Output:** fuzzy memberships u_{ik} (soft assignment) and centroids; hard labels come from $\arg\max_k u_{ik}$.

```

class FuzzyCMeans:
    """
    Fuzzy C-Means (FCM) clustering - implemented from scratch with NumPy.

    Each sample belongs to every cluster with a membership degree  $u_{ik}$  in  $[0, 1]$ ,
    and the memberships over the  $K$  clusters sum to 1 per sample.

    Parameters
    -----
    n_clusters : number of fuzzy clusters  $K$ 
    m           : fuzzifier exponent ( $m > 1$ ; larger  $m \Rightarrow$  fuzzier memberships)
    max_iter    : maximum number of alternating-optimization iterations
    tol         : convergence tolerance on the maximum membership change
    random_state : seed for reproducible random initialization
    """

    def __init__(self, n_clusters=5, m=2.0, max_iter=150, tol=1e-5, random_state=42):
        self.n_clusters = n_clusters
        self.m = m

```

```

self.max_iter = max_iter
self.tol = tol
self.random_state = random_state

def fit(self, X):
    rng = np.random.RandomState(self.random_state)
    n_samples = X.shape[0]

    # Step 1: initialize the membership matrix U randomly and normalize every row to sum to 1
    U = rng.rand(n_samples, self.n_clusters)
    U = U / U.sum(axis=1, keepdims=True)

    # Step 2: alternate between centroid and membership updates until convergence
    for iteration in range(self.max_iter):
        U_old = U.copy()

        # 2a: centroid update - membership-weighted mean of the data
        Um = U ** self.m #  $u_{ik}^m$ 
        centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]

        # 2b: compute the (n_samples x n_clusters) distance matrix to every centroid
        dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :], axis=2)
        dist = np.fmax(dist, 1e-10) # avoid division by zero

        # 2c: membership update - closer points receive higher membership
        #  $u_{ik} = 1 / \sum_j (d_{ik} / d_{ij})^{2/(m-1)}$ 
        inv_dist = 1.0 / dist
        power = 2.0 / (self.m - 1.0)
        inv_dist_p = inv_dist ** power
        U = inv_dist_p / inv_dist_p.sum(axis=1, keepdims=True)

        # 2d: stop when memberships stop changing (convergence)
        if np.max(np.abs(U - U_old)) < self.tol:
            break

    # Store results: final centroids, full membership matrix, and hard labels
    self.cluster_centers_ = centers
    self.u_ = U
    self.labels_ = np.argmax(U, axis=1) # hard assignment = cluster with maximum membership
    return self

# Fit FCM with the same K as the other algorithms (m = 2 is the standard fuzzifier)
fcm = FuzzyCMeans(n_clusters=optimal_k, m=2.0, random_state=42)
fcm.fit(X_mall_scaled)
fcm_labels = fcm.labels_
fcm_centers = scaler.inverse_transform(fcm.cluster_centers_)

# ---- Evaluation of the hard partition induced by the fuzzy memberships ----
print(f"FCM Silhouette Score: {silhouette_score(X_mall_scaled, fcm_labels):.4f}")
print(f"FCM Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, fcm_labels):.4f}")

```

```

FCM Silhouette Score: 0.5547
FCM Davies-Bouldin Index: 0.5722

```

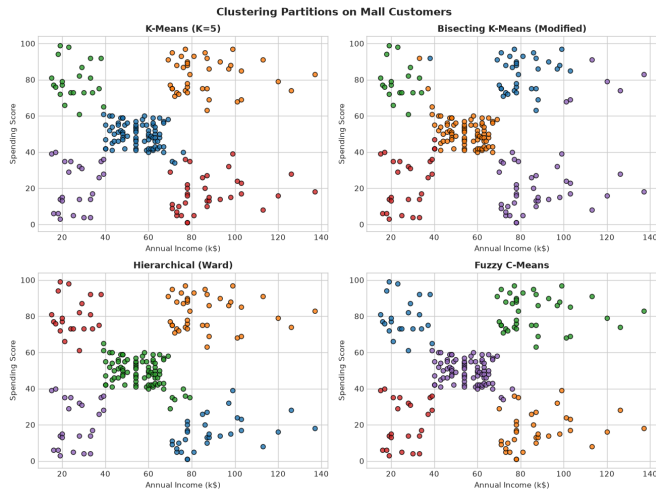
```

# ---- One combined view of all four clustering partitions ----
fig, axes = plt.subplots(2, 2, figsize=(12, 9))
partitions = [
    ("K-Means (K=5)", kmeans_labels),
    ("Bisecting K-Means (Modified)", bisecting_labels),
    ("Hierarchical (Ward)", agg_labels),
    ("Fuzzy C-Means", fcm_labels),
]
for ax, (title, labels) in zip(axes.flat, partitions):
    for cluster_id in range(optimal_k):
        pts = X_mall[labels == cluster_id]
        ax.scatter(pts[:, 0], pts[:, 1], s=35, edgecolors='k', alpha=0.8)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel("Annual Income (k$)")
    ax.set_ylabel("Spending Score")

plt.suptitle("Clustering Partitions on Mall Customers", fontsize=14, fontweight='bold')

```

```
plt.tight_layout()
plt.show()
```



5. Summary

All four algorithms are evaluated on the standardized features with the same three unsupervised metrics (Silhouette higher = better, Davies-Bouldin lower = better, Calinski-Harabasz higher = better).

```
# ---- Collect the Labels produced by the four algorithms ----
models = {
    "Standard K-Means": kmeans_labels,
    "Bisecting K-Means": bisecting_labels,
    "Hierarchical (Ward)": agg_labels,
    "Fuzzy C-Means": fcm_labels
}

# ---- Compute the three unsupervised quality metrics for each partition ----
results = []
for name, labels in models.items():
    results.append({
        "Algorithm": name,
        "Silhouette Score": silhouette_score(X_mall_scaled, labels),           # higher is better
        "Davies-Bouldin Index": davies_bouldin_score(X_mall_scaled, labels), # lower is better
        "Calinski-Harabasz Score": calinski_harabasz_score(X_mall_scaled, labels) # higher is better
    })

df_comparison = pd.DataFrame(results).set_index("Algorithm")
print("Clustering Performance Comparison on Mall Customers:")
# Highlight the best value of each metric in green for quick comparison
display(df_comparison.style.highlight_max(subset=['Silhouette Score', 'Calinski-Harabasz Score'],
color='lightgreen')
        .highlight_min(subset=['Davies-Bouldin Index'], color='lightgreen'))
```

Clustering Performance Comparison on Mall Customers:

	Silhouette Score	Davies-Bouldin Index	Calinski-Harabasz Score
Algorithm			
Standard K-Means	0.554657	0.572236	248.649320
Bisecting K-Means	0.480684	0.678789	150.938872
Hierarchical (Ward)	0.553809	0.577862	244.410326
Fuzzy C-Means	0.554657	0.572236	248.649320

02. Density-Based Learning: DBSCAN and HDBSCAN

Algorithms implemented: DBSCAN and HDBSCAN (hierarchical density-based clustering).

Dataset: Synthetic non-convex benchmark (moons + circles + blob + uniform noise).

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms + centroid baseline ----
from sklearn.cluster import DBSCAN, HDBSCAN, KMeans
# ---- Synthetic dataset generators for non-convex shapes ----
from sklearn.datasets import make_moons, make_blobs, make_circles
# ---- Nearest-neighbor search (used to pick epsilon) and scaling ----
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score, adjusted_rand_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Density-based libraries loaded successfully!

```
# ---- Build a composite benchmark dataset with complex geometry ----
# Two interleaving half-moons (non-convex)
n_samples = 600
X_moons, y_moons = make_moons(n_samples=250, noise=0.06, random_state=42)
# Two concentric circles, shifted to the right so they do not overlap the moons
X_circle, y_circle = make_circles(n_samples=200, noise=0.04, factor=0.4, random_state=42)
X_circle = X_circle * 0.7 + np.array([2.2, 0.5])
# One sparse Gaussian blob above the moons
X_blob, y_blob = make_blobs(n_samples=100, centers=[[0.5, 2.0]], cluster_std=0.2, random_state=42)
# Uniform background noise (outliers) spanning the whole area
X_noise = np.random.uniform(low=[-1.5, -1.0], high=[3.2, 2.8], size=(50, 2))

# Stack all point groups into one feature matrix
X_raw = np.vstack([X_moons, X_circle, X_blob, X_noise])
# Ground-truth labels for benchmarking (noise points are marked with -1)
y_true = np.concatenate([y_moons, y_circle + 2, y_blob + 4, -np.ones(50, dtype=int)])

# Standardize so that epsilon (a distance threshold) has a comparable meaning in both axes
scaler = StandardScaler()
X = scaler.fit_transform(X_raw)

# ---- Plot the raw benchmark to show the shapes/noise the algorithms must handle ----

print(f"Benchmark data: {X.shape[0]} points, {X.shape[1]} features")
print(f"Ground-truth groups: {len(set(y_true[y_true != -1]))} + noise")
```

Benchmark data: 600 points, 2 features
Ground-truth groups: 5 + noise

1. DBSCAN

Step-by-Step Algorithm (DBSCAN)

1. **Parameters:** radius ϵ and minimum points MinPts (here 0.22 and 6).
2. For every point count its ϵ -neighborhood; points with $\geq$ MinPts neighbors become **core points**.
3. **Grow clusters:** connect core points that lie within ϵ of each other (density-reachability chain).
4. Attach border points (within ϵ of a core point) to that core's cluster.
5. Label every point that is neither core nor border as **noise (-1)**.
6. **Output:** arbitrary-shaped clusters + explicit noise labels. No K is required.

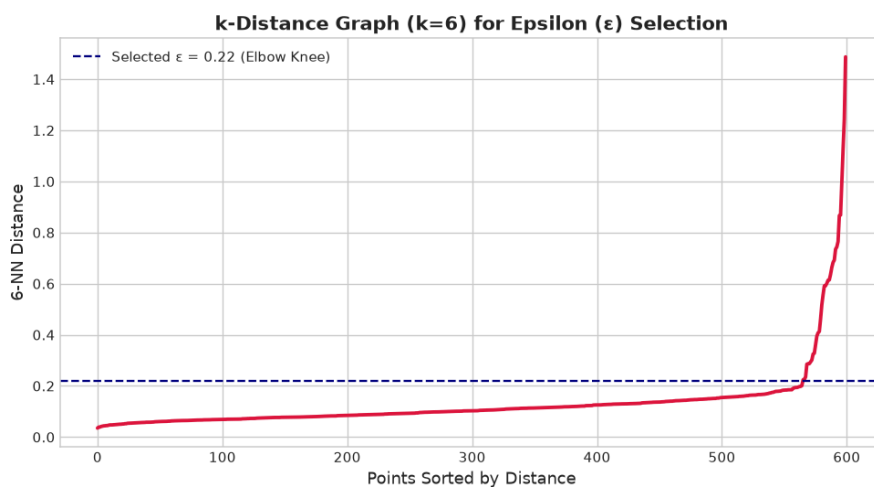
How ϵ and MinPts Are Chosen Here

1. Set MinPts $\approx 2 \times$ dimensions.
2. Plot the sorted distance to the MinPts-th neighbor and pick the "knee" as ϵ .
3. Fit DBSCAN and inspect cluster count, noise %, and agreement with the ground truth.

```
# ---- Choosing epsilon from the k-distance graph ----
# Heuristic: sort the distance of each point to its k-th nearest neighbor.
# The "knee" of the curve marks the transition from dense clusters to sparse noise,
# which is a good value for epsilon.
min_samples = 6 # MinPts rule of thumb: at least 2 * number of dimensions
nbrs = NearestNeighbors(n_neighbors=min_samples).fit(X)
distances, indices = nbrs.kneighbors(X) # distance to each point's k neighbors

k_distances = np.sort(distances[:, -1]) # k-th neighbor distance, sorted ascending

plt.figure(figsize=(9, 5))
plt.plot(k_distances, color='crimson', linewidth=2.5)
plt.title(f"k-Distance Graph (k={min_samples}) for Epsilon ( $\epsilon$ ) Selection", fontsize=14, fontweight='bold')
plt.xlabel("Points Sorted by Distance", fontsize=12)
plt.ylabel(f"{min_samples}-NN Distance", fontsize=12)
plt.axhline(y=0.22, color='navy', linestyle='--', label='Selected  $\epsilon = 0.22$  (Elbow Knee)')
plt.legend()
plt.tight_layout()
plt.show()
```



```
# ---- Fit DBSCAN and visualize the partition ----
eps_selected = 0.22
dbscan = DBSCAN(eps=eps_selected, min_samples=min_samples)
db_labels = dbscan.fit_predict(X)

# Core points are the density anchors; all remaining non-noise points are border points
core_samples_mask = np.zeros_like(db_labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True
```

```

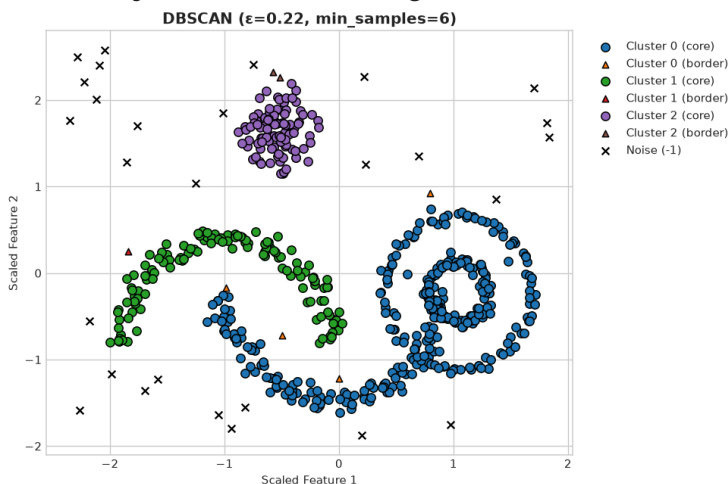
n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise_db = np.sum(db_labels == -1)
print(f"DBSCAN: {n_clusters_db} clusters, {n_noise_db} noise points ({n_noise_db / len(X) * 100:.1f}%)")
print(f"DBSCAN Adjusted Rand Index vs ground truth: {adjusted_rand_score(y_true, db_labels):.4f}")

plt.figure(figsize=(9, 6))
for k in set(db_labels):
    mask = db_labels == k
    if k == -1:
        plt.scatter(X[mask, 0], X[mask, 1], c='black', marker='x', s=40, label='Noise (-1)')
    else:
        plt.scatter(X[mask & core_samples_mask, 0], X[mask & core_samples_mask, 1], s=55, edgecolors='k',
                    label=f'Cluster {k} (core)')
        plt.scatter(X[mask & ~core_samples_mask, 0], X[mask & ~core_samples_mask, 1], s=30, marker='^',
                    edgecolors='k', label=f'Cluster {k} (border)')

plt.title(f"DBSCAN ( $\epsilon$ ={eps_selected}, min_samples={min_samples})", fontsize=13, fontweight='bold')
plt.xlabel("Scaled Feature 1")
plt.ylabel("Scaled Feature 2")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

```

DBSCAN: 3 clusters, 28 noise points (4.7%)
 DBSCAN Adjusted Rand Index vs ground truth: 0.4637



2. HDBSCAN

Step-by-Step Algorithm (HDBSCAN)

1. Compute the **core distance** $d_{\text{core}}(p)$ = distance from p to its k -th nearest neighbor.
2. Define the **mutual reachability distance**: $d_{\text{mreach}}(a,b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a,b)\}$.
3. Build a **minimum spanning tree** on the mutual-reachability graph.
4. Convert the MST into a **cluster hierarchy** and then into a **condensed tree** (cluster births/deaths).
5. Score each candidate cluster by **stability** $S(C) = \sum (\lambda_p - \lambda_{\text{birth}}(C))$ and keep the most stable ones.
6. **Output**: clusters of varying density, membership probabilities, and noise labels - with no global ϵ .

```

# ---- Fit HDBSCAN (no global epsilon needed) and visualize ----
hdb = HDBSCAN(min_cluster_size=15, min_samples=6, store_centers='centroid', copy=False)
hdb_labels = hdb.fit_predict(X)

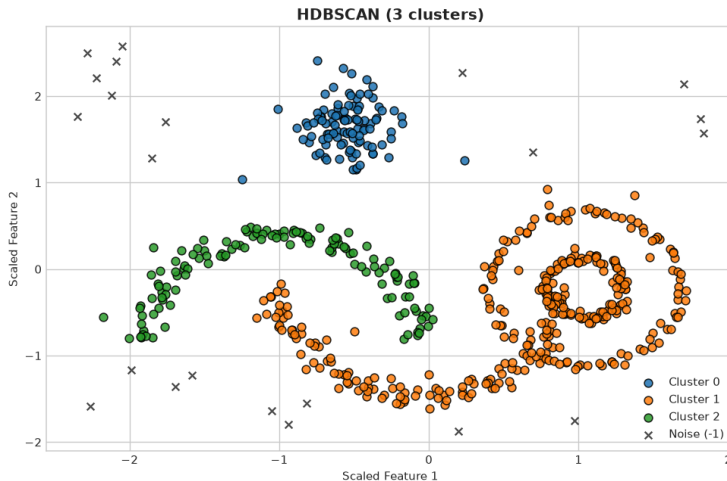
n_clusters_hdb = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
n_noise_hdb = np.sum(hdb_labels == -1)
print(f"HDBSCAN: {n_clusters_hdb} clusters, {n_noise_hdb} noise points ({n_noise_hdb / len(X) * 100:.1f}%)")
print(f"HDBSCAN Adjusted Rand Index vs ground truth: {adjusted_rand_score(y_true, hdb_labels):.4f}")

plt.figure(figsize=(9, 6))
palette = sns.color_palette("tab10", max(n_clusters_hdb, 1))
for cluster_id in range(n_clusters_hdb):

```

```
pts = X[hdb_labels == cluster_id]
plt.scatter(pts[:, 0], pts[:, 1], color=palette[cluster_id], edgecolors='k', s=50, label=f'Cluster
{cluster_id}', alpha=0.85)
noise_pts = X[hdb_labels == -1]
plt.scatter(noise_pts[:, 0], noise_pts[:, 1], color='black', marker='x', s=40, label='Noise (-1)',
alpha=0.7)
plt.title(f"HDBSCAN ({n_clusters_hdb} clusters)", fontsize=13, fontweight='bold')
plt.xlabel("Scaled Feature 1")
plt.ylabel("Scaled Feature 2")
plt.legend()
plt.tight_layout()
plt.show()
```

HDBSCAN: 3 clusters, 22 noise points (3.7%)
HDBSCAN Adjusted Rand Index vs ground truth: 0.4578



3. Summary

Both density-based algorithms find arbitrary-shaped clusters and label sparse points as noise; DBSCAN requires a global ϵ while HDBSCAN selects clusters across density levels.

```
# ---- Summary metrics (silhouette computed on non-noise points only) ----
def density_metrics(name, labels):
    mask = labels != -1
    sil = silhouette_score(X[mask], labels[mask]) if len(set(labels[mask])) > 1 else np.nan
    return {
        "Algorithm": name,
        "Clusters Found": len(set(labels)) - (1 if -1 in labels else 0),
        "Noise Points (%)": f"{np.sum(labels == -1) / len(labels) * 100:.1f}%",
        "Adjusted Rand Index": adjusted_rand_score(y_true, labels),
        "Silhouette (Non-Noise)": sil,
    }

comparison_table = pd.DataFrame([
    density_metrics("DBSCAN", db_labels),
    density_metrics("HDBSCAN", hdb_labels),
]).set_index("Algorithm")
display(comparison_table.style.highlight_max(subset=['Adjusted Rand Index', 'Silhouette (Non-Noise)'],
color='lightgreen'))
```

	Clusters Found	Noise Points (%)	Adjusted Rand Index	Silhouette (Non-Noise)
Algorithm				
DBSCAN	3	4.7%	0.463683	0.434245
HDBSCAN	3	3.7%	0.457798	0.433431

03. Semi-Supervised Learning: Self-Training

Algorithm implemented: Self-Training Classifier (iterative pseudo-labeling) with an RBF-kernel SVC base estimator.

Dataset: Breast Cancer Wisconsin - only 15% of the training labels are kept; the rest are treated as unlabeled.

```
# ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
import warnings
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + train/test utilities ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Semi-supervised self-training wrapper ----
from sklearn.semi_supervised import SelfTrainingClassifier
# ---- Base estimators compared inside self-training ----
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
confusion_matrix, classification_report

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# The installed scikit-learn still supports SVC(probability=True) but warns; silence that specific notice
warnings.filterwarnings("ignore", message="The `probability` parameter was deprecated",
category=FutureWarning)
np.random.seed(42)
print("Semi-supervised libraries imported successfully!")

# ---- Load the Breast Cancer Wisconsin dataset ----
data = load_breast_cancer()
X_full = data.data          # 30 continuous diagnostic features
y_full = data.target        # 0 = malignant, 1 = benign
feature_names = data.feature_names
target_names = data.target_names

# ---- Train / Test split (stratified to preserve class balance) ----
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
)

# ---- Standardize features (fit on train only to avoid test leakage) ----
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_test_scaled = scaler.transform(X_test)

# ---- Simulate the semi-supervised setting: hide 85% of the training labels ----
unlabeled_ratio = 0.85
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio    # True -> mask

y_train_semi = np.copy(y_train_full)
y_train_semi[random_unlabeled_points] = -1    # -1 is the scikit-learn convention for "unlabeled"

n_labeled = np.sum(y_train_semi != -1)
n_unlabeled = np.sum(y_train_semi == -1)
```

```

print(f"Total Training Samples:    {len(y_train_full)}")
print(f" - Labeled Samples (15%): {n_labeled}")
print(f" - Unlabeled Samples (85%): {n_unlabeled}")
print(f"Holdout Test Samples:        {len(y_test)}")

```

```

Semi-supervised libraries imported successfully!
Total Training Samples:    398
  - Labeled Samples (15%): 61
  - Unlabeled Samples (85%): 337
Holdout Test Samples:      171

```

Step-by-Step Algorithm (Self-Training)

1. **Input:** labeled set L , unlabeled pool U , base classifier f , confidence threshold τ .
2. Train f on the current labeled set L .
3. Predict class probabilities for every sample in U .
4. Add predictions with max probability $\geq \tau$ to L as pseudo-labels; remove them from U .
5. Repeat 2-4 until no new pseudo-labels are added or the iteration limit is reached.
6. Return the final classifier; evaluate once on the untouched test set.

Pseudocode (Self-Training Classifier)

```

L = labeled set          # 15% of training data
U = unlabeled pool       # 85% of training data (labels hidden)
repeat until no new confident labels or max_iter reached:
    f = train(base_estimator, L)          # e.g. SVC or Random Forest
    P = f.predict_proba(U)                # class probabilities for each unlabeled
point
    S = { (x, argmax P(x)) : max P(x) >= threshold } # confident pseudo-labels
    L = L union S                               # augment labeled set
    U = U minus S                               # remove newly labeled points
return f

```

Key hyperparameter: the confidence threshold τ controls the purity/quantity trade-off of pseudo-labels.

```

# ---- Experiment 1: Self-Training with an RBF-kernel SVC ----

# 1. Supervised baseline trained ONLY on the ~60 Labeled samples
mask_labeled = (y_train_semi != -1)          # boolean mask of the Labeled subset
X_labeled = X_train_full_scaled[mask_labeled]
y_labeled = y_train_semi[mask_labeled]

base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
base_svc.fit(X_labeled, y_labeled)
y_pred_baseline_svc = base_svc.predict(X_test_scaled)
acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)

# 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above threshold 0.80
self_training_svc = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=0.80,          # only accept pseudo-labels with confidence >= 0.80
    criterion='threshold',   # use the confidence-threshold selection rule
    max_iter=15,
    verbose=True             # print how many labels are added per iteration
)
self_training_svc.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
f1_semi_svc = f1_score(y_test, y_pred_semi_svc)

# 3. Fully supervised "ceiling": train on 100% of the ground-truth labels
full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
full_svc.fit(X_train_full_scaled, y_train_full)
y_pred_full_svc = full_svc.predict(X_test_scaled)

```

```

acc_full_svc = accuracy_score(y_test, y_pred_full_svc)
f1_full_svc = f1_score(y_test, y_pred_full_svc)

# ---- Compare the three settings ----
print(f"\n--- SVC Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 = {f1_baseline_svc:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 = {f1_semi_svc:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 = {f1_full_svc:.4f}")

```

End of iteration 1, added 292 new labels.
End of iteration 2, added 22 new labels.
End of iteration 3, added 5 new labels.
End of iteration 4, added 2 new labels.

```

--- SVC Evaluation Results ---
Supervised Baseline (15% labeled): Accuracy = 0.9298, F1 = 0.9469
Self-Training (Semi-supervised): Accuracy = 0.9415, F1 = 0.9554
Fully Supervised Ceiling (100%): Accuracy = 0.9766, F1 = 0.9813

```

```

# ---- Inspect how self-training converged ----
# n_iter_ : number of self-training iterations performed
# termination_condition_ : why training stopped (e.g. 'no_change' = no new confident labels)
# transduction_ : final labels for every training sample (-1 = still unlabeled)
n_iter = self_training_svc.n_iter_
termination_condition = self_training_svc.termination_condition_
labeled_final_mask = (self_training_svc.transduction_ != -1)

print(f"Self-Training Convergence Summary:")
print(f" - Iterations completed: {n_iter}")
print(f" - Termination condition: {termination_condition}")
print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)} / {len(y_train_full)}")
print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")

```

Self-Training Convergence Summary:

- Iterations completed: 5
- Termination condition: no_change
- Total samples labeled after self-training: 382 / 398
- Pseudo-labels newly added: 321

Results

Compare the 15%-labeled baseline, self-training, and the fully supervised ceiling.

```

# ---- Compact comparison table (SVC) ----
summary_data = [
    {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc, "F1-Score": f1_baseline_svc,
     "Training Samples": n_labeled},
    {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc, "F1-Score": f1_semi_svc,
     "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
    {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc, "F1-Score": f1_full_svc,
     "Training Samples": len(y_train_full)},
]
summary_df = pd.DataFrame(summary_data).set_index("Model")
display(summary_df.style.highlight_max(subset=['Accuracy', 'F1-Score'], color='lightgreen'))

```

	Accuracy	F1-Score	Training Samples
Model			
SVC (Baseline 15% Labeled)	0.929825	0.946903	61
SVC (Self-Training Semi-Supervised)	0.941520	0.955357	382
SVC (Fully Supervised Ceiling 100%)	0.976608	0.981308	398

04. Ensemble Learning

Algorithms implemented: Random Forest Regression (RFR), Random Forest Classification (RFC), XGBoost, AdaBoost, CatBoost.

Datasets: California Housing (regression) and Heart Disease (`data/heart_disease.csv` , classification).

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score,
f1_score, roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)

# ---- Load the California Housing dataset (real-world regression benchmark) ----
housing = fetch_california_housing(as_frame=True)
# Sample 5,000 rows so every ensemble trains quickly while remaining representative
df_reg = housing.frame.sample(n=5000, random_state=42)

X_reg = df_reg[housing.feature_names] # 8 socio-economic / geographic features
y_reg = df_reg['MedHouseVal'] # target: median house value

# ---- Train/test split (80% / 20%) ----
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.20, random_state=42
)

print(f"Regression Training Samples: {X_train_reg.shape[0]}, Test Samples: {X_test_reg.shape[0]}")
df_reg.head()
```

Regression Training Samples: 4000, Test Samples: 1000

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
20046	1.6812	25.0	4.192201	1.022284	1392.0	3.877437	36.06	-119.01	0.47700
3024	2.5313	30.0	5.039384	1.193493	1565.0	2.679795	35.14	-119.46	0.45800
15663	3.4801	52.0	3.977155	1.185877	1310.0	1.360332	37.80	-122.44	5.00001
20484	5.7376	17.0	6.163636	1.020202	1705.0	3.444444	34.28	-118.72	2.18600
9814	3.7250	34.0	5.492991	1.028037	1063.0	2.483645	36.62	-121.93	2.78000

1. Random Forest Regression (RFR)

Step-by-Step Algorithm (Random Forest Regression)

1. Draw B bootstrap samples from the training data (sampling with replacement).
2. Grow one decision tree per bootstrap sample; at each split consider only a random subset of features.

3. Let every tree predict independently on the test point.
4. Average the B tree predictions: $\hat{f}_{RF}(x) = (1/B) \sum T_b(x)$.
5. Use out-of-bag samples (not drawn in a bootstrap) to estimate generalization (OOB score).

```
# ---- Random Forest Regression (bagging of decision trees) ----
# n_estimators = number of trees, oob_score = estimate accuracy from out-of-bag samples
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True, random_state=42, n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_rf = rf_reg.predict(X_test_reg)
r2_rf = r2_score(y_test_reg, y_pred_rf)
rmse_rf = root_mean_squared_error(y_test_reg, y_pred_rf)
mae_rf = mean_absolute_error(y_test_reg, y_pred_rf)

# explained variance (1.0 = perfect)
# error in target units
# robust average error

print(f"Random Forest Regressor Results:")
print(f" - Out-of-Bag (OOB) R² Score: {rf_reg.oob_score_:.4f}")
print(f" - Test R² Score: {r2_rf:.4f}")
print(f" - Test RMSE: {rmse_rf:.4f}")
print(f" - Test MAE: {mae_rf:.4f}")
```

Random Forest Regressor Results:

- Out-of-Bag (OOB) R² Score: 0.7513
- Test R² Score: 0.7415
- Test RMSE: 0.5908
- Test MAE: 0.3999

2. XGBoost Regression

Step-by-Step Algorithm (XGBoost Regression)

1. Start with a constant prediction (e.g. the mean target).
2. For each boosting round t: a. Compute first-order gradients g_i and second-order Hessians h_i of the loss. b. Grow a new tree that minimizes the regularized objective: $\sum [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$. c. $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$ penalizes tree complexity (T leaves) and leaf weights.
3. Add the new tree to the ensemble with a shrinkage factor (learning_rate).
4. Repeat steps 2–3 for n_estimators rounds; prediction = sum of all trees.

```
# ---- XGBoost Regression (gradient boosting with 2nd-order optimization) ----
# subsample/colsample add stochasticity; reg_alpha/reg_lambda are L1/L2 regularizers.
xgb_reg = xgb.XGBRegressor(
    n_estimators=150,
    learning_rate=0.08,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,
    reg_lambda=1.0,
    random_state=42
)
xgb_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_xgb = xgb_reg.predict(X_test_reg)
r2_xgb = r2_score(y_test_reg, y_pred_xgb)
rmse_xgb = root_mean_squared_error(y_test_reg, y_pred_xgb)
mae_xgb = mean_absolute_error(y_test_reg, y_pred_xgb)

print(f"XGBoost Regressor Results:")
print(f" - Test R² Score: {r2_xgb:.4f}")
print(f" - Test RMSE: {rmse_xgb:.4f}")
print(f" - Test MAE: {mae_xgb:.4f}")
```

XGBoost Regressor Results:

- Test R² Score: 0.8025

```
- Test RMSE:      0.5165
- Test MAE:       0.3467
```

3. AdaBoost Regression

Step-by-Step Algorithm (AdaBoost Regression)

1. Initialize equal weights w_i for all training samples.
2. Train a weak regressor on the weighted data.
3. Compute the relative error $e_i = |y_i - \hat{y}_m(x_i)| / \max_j |y_j - \hat{y}_m(x_j)|$.
4. Set the learner weight $\beta_m = \bar{L}_m / (1 - \bar{L}_m)$ from the average weighted loss.
5. Increase weights of poorly predicted samples: $w_i \leftarrow w_i \cdot \beta_m^{(1 - e_i)}$.
6. Repeat steps 2–5 for M rounds, then take the weighted median of all predictions.

```
# ---- AdaBoost Regression (sequential re-weighting of weak regressors) ----
# Each new weak learner focuses more on the samples the previous ones predicted poorly.
ada_reg = AdaBoostRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
ada_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_ada = ada_reg.predict(X_test_reg)
r2_ada = r2_score(y_test_reg, y_pred_ada)
rmse_ada = root_mean_squared_error(y_test_reg, y_pred_ada)
mae_ada = mean_absolute_error(y_test_reg, y_pred_ada)

print(f"AdaBoost Regressor Results:")
print(f" - Test R² Score: {r2_ada:.4f}")
print(f" - Test RMSE:      {rmse_ada:.4f}")
print(f" - Test MAE:       {mae_ada:.4f}")
```

```
AdaBoost Regressor Results:
- Test R² Score: 0.5888
- Test RMSE:     0.7452
- Test MAE:      0.5842
```

4. CatBoost Regression

Step-by-Step Algorithm (CatBoost Regression)

1. Build **oblivious (symmetric) trees**: the same split condition is used at every node of a depth level.
2. Use **ordered boosting**: train each tree on a random permutation of the data to avoid target leakage/prediction shift.
3. Compute leaf values from the ordered statistics, then add the tree to the ensemble with the learning rate.
4. Repeat for the configured number of iterations.
5. Predict by summing the values of all trees for the input.

```
# ---- CatBoost Regression (ordered boosting + oblivious trees) ----
# l2_leaf_reg penalizes large leaf values; verbose=False keeps the notebook output clean.
cb_reg = cb.CatBoostRegressor(
    iterations=200,
    learning_rate=0.08,
    depth=6,
    l2_leaf_reg=3.0,
    random_seed=42,
    verbose=False
)
cb_reg.fit(X_train_reg, y_train_reg)

# ---- Predict and evaluate ----
y_pred_cb = cb_reg.predict(X_test_reg)
r2_cb = r2_score(y_test_reg, y_pred_cb)
rmse_cb = root_mean_squared_error(y_test_reg, y_pred_cb)
```

```
mae_cb = mean_absolute_error(y_test_reg, y_pred_cb)
```

```
print(f"CatBoost Regressor Results:")
print(f" - Test R2 Score: {r2_cb:.4f}")
print(f" - Test RMSE:      {rmse_cb:.4f}")
print(f" - Test MAE:        {mae_cb:.4f}")
```

CatBoost Regressor Results:

- Test R² Score: 0.8029
- Test RMSE: 0.5160
- Test MAE: 0.3535

Regression Summary

```
# ---- Side-by-side regression comparison ----
reg_comparison = pd.DataFrame([
    {"Model": "Random Forest Regressor (RFR)", "R2 Score": r2_rf, "RMSE": rmse_rf, "MAE": mae_rf},
    {"Model": "AdaBoost Regressor", "R2 Score": r2_ada, "RMSE": rmse_ada, "MAE": mae_ada},
    {"Model": "XGBoost Regressor", "R2 Score": r2_xgb, "RMSE": rmse_xgb, "MAE": mae_xgb},
    {"Model": "CatBoost Regressor", "R2 Score": r2_cb, "RMSE": rmse_cb, "MAE": mae_cb}
]).set_index("Model")

print("Ensemble Regression Comparison on California Housing:")
# Green = best value (max R2, min RMSE/MAE)
display(reg_comparison.style.highlight_max(subset=['R2 Score'], color='lightgreen')
        .highlight_min(subset=['RMSE', 'MAE'], color='lightgreen'))
```

Ensemble Regression Comparison on California Housing:

	R ² Score	RMSE	MAE
Model			
Random Forest Regressor (RFR)	0.741531	0.590846	0.399898
AdaBoost Regressor	0.588827	0.745216	0.584228
XGBoost Regressor	0.802459	0.516534	0.346691
CatBoost Regressor	0.802873	0.515992	0.353549

5. Random Forest Classification (RFC)

Step-by-Step Algorithm (Random Forest Classification)

1. Draw B bootstrap samples and grow a tree per sample (random feature subset at each split).
2. Each tree casts a unit vote for the class at input x.
3. Final prediction = majority vote: $\hat{C}_{RF}(x) = \text{argmax}_k \sum_b I(T_b(x) = k)$.
4. Class probabilities = fraction of trees voting for each class (used for ROC-AUC).

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")
```

```
# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Heart Disease dataset: 303 rows x 14 columns
Classification Training Set: (227, 13), Test Set: (76, 13)
Random Forest Classifier: Accuracy = 0.7763, F1 = 0.8046, ROC-AUC = 0.8704

05. Multilayer Perceptron (MLP)

Algorithms implemented: MLP with Scikit-Learn (`MLPClassifier`) and a custom PyTorch deep MLP.

Dataset: Handwritten digits (8x8 grayscale images, 64 features, 10 classes).

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + preprocessing/metrics ----
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Scikit-Learn MLP implementation ----
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ---- PyTorch for the custom deep MLP ----
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix seeds for reproducible training
torch.manual_seed(42)
np.random.seed(42)
print(f"PyTorch version: {torch.__version__}")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # use GPU if available
print(f"Computation Device: {device}")

# ---- Load the handwritten digits dataset (8x8 grayscale images) ----
digits = load_digits()
X_raw = digits.data # flattened images: 1797 samples x 64 pixel features
y = digits.target # digit class 0..9

print(f"Dataset Shape: {X_raw.shape}, Target Classes: {np.unique(y)}")
# ---- Train / Validation / Test split (70% / 15% / 15%) ----
# First cut off the test set...
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X_raw, y, test_size=0.15, random_state=42, stratify=y
)
# ...then split the remainder into train and validation
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.1765, random_state=42, stratify=y_train_val
)

# ---- Feature scaling: fit on training data only (avoid validation/test Leakage) ----
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)

print(f"Training samples: {X_train_scaled.shape[0]}")
print(f"Validation samples: {X_val_scaled.shape[0]}")
print(f"Test samples: {X_test_scaled.shape[0]}")
```

PyTorch version: 2.14.0+cpu

Computation Device: cpu

Dataset Shape: (1797, 64), Target Classes: [0 1 2 3 4 5 6 7 8 9]

Training samples: 1257
Validation samples: 270
Test samples: 270

1. Scikit-Learn MLP

Step-by-Step Algorithm (Multilayer Perceptron)

1. **Initialize** weights W and biases b for every layer (random small values).
2. **Forward pass:** for layer l , compute $a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$; the last layer produces class logits.
3. **Loss:** convert logits to probabilities with softmax and compute cross-entropy (plus L2 penalty).
4. **Backward pass:** use the chain rule (backpropagation) to compute the gradient of the loss w.r.t. every weight.
5. **Update:** adjust weights with an optimizer (Adam): $W \leftarrow W - \eta \cdot (\text{gradient} + \text{regularization term})$.
6. **Repeat** steps 2–5 for many epochs over mini-batches until the loss converges.
7. **Predict:** take argmax of the output probabilities for each test sample.

Scikit-Learn Training Process Used Here

1. Build a $64 \rightarrow 128 \rightarrow 64 \rightarrow 10$ network with ReLU activations.
2. Optimize with Adam and L2 regularization (`alpha`).
3. Use early stopping: monitor a validation split and stop when it stops improving.
4. Record the loss curve and evaluate once on the untouched test set.

```
# ---- Scikit-Learn MLPClassifier: 64 -> 128 -> 64 -> 10 ----
mlp_sklearn = MLPClassifier(
    hidden_layer_sizes=(128, 64),  # two hidden layers
    activation='relu',
    solver='adam',                  # adaptive gradient optimizer
    alpha=0.001,                   # L2 regularization strength
    batch_size=64,
    learning_rate_init=0.005,
    max_iter=150,
    early_stopping=True,           # stop when validation score stops improving
    validation_fraction=0.15,     # internal validation split for early stopping
    random_state=42
)

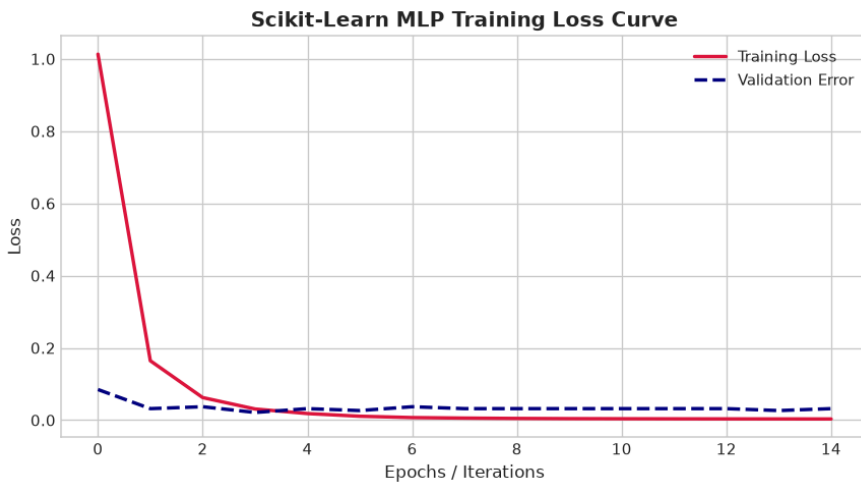
# Train (forward pass + backpropagation happen inside .fit)
mlp_sklearn.fit(X_train_scaled, y_train)

# Evaluate on the untouched test set
y_pred_sk = mlp_sklearn.predict(X_test_scaled)
acc_sk = accuracy_score(y_test, y_pred_sk)

print(f"Scikit-Learn MLP Test Accuracy: {acc_sk:.4f}")
print(f"Converged after {mlp_sklearn.n_iter_} iterations")

# ---- Plot the training loss curve recorded by scikit-Learn ----
plt.figure(figsize=(8, 4.5))
plt.plot(mlp_sklearn.loss_curve_, color='crimson', lw=2.2, label='Training Loss')
if hasattr(mlp_sklearn, 'validation_scores_'):
    # validation_scores_ stores accuracy; convert to error for a comparable scale
    plt.plot([1 - s for s in mlp_sklearn.validation_scores_], color='navy', lw=2.2, linestyle='--',
             label='Validation Error')
plt.title("Scikit-Learn MLP Training Loss Curve", fontsize=13, fontweight='bold')
plt.xlabel("Epochs / Iterations", fontsize=11)
plt.ylabel("Loss", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()
```

Scikit-Learn MLP Test Accuracy: 0.9630
Converged after 15 iterations



2. Custom PyTorch MLP

PyTorch Training Process (explicit training loop)

1. **Data:** wrap scaled features/labels in Tensors and DataLoaders (mini-batches + shuffling).
2. **Architecture:** Linear → BatchNorm → ReLU → Dropout, twice, then a 10-unit output layer.
3. **Configure:** CrossEntropyLoss + Adam (weight decay) + ReduceLROnPlateau scheduler.
4. **For each epoch:** forward pass, compute loss, backpropagate, update weights.
5. **Monitor:** evaluate on the validation set after every epoch (no gradient updates).
6. **After training:** evaluate once on the test set, then inspect the confusion matrix and sample predictions.

```
# ---- Convert scaled NumPy arrays into PyTorch tensors + DataLoaders ----
train_dataset = TensorDataset(torch.tensor(X_train_scaled, dtype=torch.float32), torch.tensor(y_train, dtype=torch.long))
val_dataset   = TensorDataset(torch.tensor(X_val_scaled, dtype=torch.float32), torch.tensor(y_val, dtype=torch.long))
test_dataset  = TensorDataset(torch.tensor(X_test_scaled, dtype=torch.float32), torch.tensor(y_test, dtype=torch.long))

# DataLoaders handle mini-batching and shuffling during training
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=64, shuffle=False)
test_loader  = DataLoader(test_dataset, batch_size=64, shuffle=False)

class DeepMLP(nn.Module):
    """Fully-connected classifier: 64 -> 128 -> 64 -> 10 with BatchNorm + Dropout."""
    def __init__(self, in_features=64, num_classes=10):
        super(DeepMLP, self).__init__()
        self.net = nn.Sequential(
            # Hidden block 1
            nn.Linear(in_features, 128),
            nn.BatchNorm1d(128), # stabilizes activations, speeds up training
            nn.ReLU(),           # non-linear activation
            nn.Dropout(0.25),    # regularization: randomly drop 25% of neurons

            # Hidden block 2
            nn.Linear(128, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.20),

            # Output layer: 10 raw logits (one per digit class)
            nn.Linear(64, num_classes)
        )

    def forward(self, x):
        return self.net(x)
```

```
model = DeepMLP().to(device) # move all parameters to CPU/GPU
print(model)
```

```
DeepMLP(
  (net): Sequential(
    (0): Linear(in_features=64, out_features=128, bias=True)
    (1): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.25, inplace=False)
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, bias=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.2, inplace=False)
    (8): Linear(in_features=64, out_features=10, bias=True)
  )
)
```

```
# ---- Training configuration ----
criterion = nn.CrossEntropyLoss() # softmax + NLL Loss
optimizer = optim.Adam(model.parameters(), lr=0.003, weight_decay=1e-4) # weight decay = L2
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=5)

epochs = 50
history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

for epoch in range(epochs):
    # ----- Training phase -----
    model.train() # enable Dropout/BatchNorm training behavior
    running_loss, correct, total = 0.0, 0, 0
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad() # reset gradients from the previous step
        outputs = model(inputs) # forward pass
        loss = criterion(outputs, labels) # compute cross-entropy
        loss.backward() # backpropagation (compute gradients)
        optimizer.step() # update weights

        # Accumulate metrics (loss is averaged over the batch, so multiply by batch size)
        running_loss += loss.item() * inputs.size(0)
        _, preds = torch.max(outputs, 1) # predicted class = highest logit
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    epoch_train_loss = running_loss / total
    epoch_train_acc = correct / total

    # ----- Validation phase -----
    model.eval() # disable Dropout, use running BatchNorm stats
    val_loss, val_correct, val_total = 0.0, 0, 0
    with torch.no_grad(): # no gradients needed for evaluation
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            val_loss += loss.item() * inputs.size(0)
            _, preds = torch.max(outputs, 1)
            val_correct += (preds == labels).sum().item()
            val_total += labels.size(0)

    epoch_val_loss = val_loss / val_total
    epoch_val_acc = val_correct / val_total
    scheduler.step(epoch_val_loss) # reduce LR if validation loss plateaus

    # Record metrics for plotting
    history['train_loss'].append(epoch_train_loss)
    history['val_loss'].append(epoch_val_loss)
    history['train_acc'].append(epoch_train_acc)
    history['val_acc'].append(epoch_val_acc)

    # Print a progress line every 10 epochs (and the first one)
    if (epoch + 1) % 10 == 0 or epoch == 0:
```

```
print(f"Epoch [{epoch+1:02d}/{epochs}] "
      f"Train Loss: {epoch_train_loss:.4f} | Train Acc: {epoch_train_acc:.4f} || "
      f"Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f}")
```

```
Epoch [01/50] Train Loss: 1.1992 | Train Acc: 0.7327 || Val Loss: 0.4583 | Val Acc: 0.9111
Epoch [10/50] Train Loss: 0.0718 | Train Acc: 0.9793 || Val Loss: 0.0717 | Val Acc: 0.9667
Epoch [20/50] Train Loss: 0.0392 | Train Acc: 0.9889 || Val Loss: 0.0410 | Val Acc: 0.9852
Epoch [30/50] Train Loss: 0.0186 | Train Acc: 0.9936 || Val Loss: 0.0397 | Val Acc: 0.9852
Epoch [40/50] Train Loss: 0.0321 | Train Acc: 0.9905 || Val Loss: 0.0409 | Val Acc: 0.9815
Epoch [50/50] Train Loss: 0.0117 | Train Acc: 0.9984 || Val Loss: 0.0317 | Val Acc: 0.9889
```

```
# ---- Final evaluation on the held-out test set ----
model.eval()
test_preds = []
test_targets = []

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs = inputs.to(device)
        outputs = model(inputs)
        _, preds = torch.max(outputs, 1)          # predicted digit
        test_preds.extend(preds.cpu().numpy())    # move back to CPU for metrics
        test_targets.extend(labels.numpy())

acc_pytorch = accuracy_score(test_targets, test_preds)
print(f"PyTorch Deep MLP Test Accuracy: {acc_pytorch:.4f}\n")
print("Detailed Classification Report:")
# Per-class precision/recall/F1 for all 10 digits
print(classification_report(test_targets, test_preds, digits=4))
```

PyTorch Deep MLP Test Accuracy: 0.9889

Detailed Classification Report:

	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	27
1	0.9286	0.9630	0.9455	27
2	1.0000	1.0000	1.0000	27
3	1.0000	1.0000	1.0000	28
4	0.9643	1.0000	0.9818	27
5	1.0000	1.0000	1.0000	27
6	1.0000	1.0000	1.0000	27
7	1.0000	1.0000	1.0000	27
8	1.0000	0.9231	0.9600	26
9	1.0000	1.0000	1.0000	27
accuracy				270
macro avg	0.9893	0.9886	0.9887	270
weighted avg	0.9893	0.9889	0.9889	270

06. Recurrent Neural Networks (RNN)

Algorithms implemented: Vanilla RNN (Elman) and LSTM in PyTorch.

Dataset: Monthly Airline Passengers (`data/airline_passengers.csv`) - 12-month sliding windows forecast the next month.

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scaling + regression metrics ----
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import root_mean_squared_error, mean_absolute_error

# ---- PyTorch for the recurrent models ----
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

# Reproducibility
torch.manual_seed(42)
np.random.seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch version: {torch.__version__} on {device}")

# ---- Load the monthly airline passengers time series ----
df_air = pd.read_csv('../data/airline_passengers.csv')
print("Dataset Head:")
display(df_air.head())

# Extract the target series as float32 (shape: N x 1)
passengers = df_air['Passengers'].values.astype(np.float32).reshape(-1, 1)
# ---- Normalize to [0, 1] (RNNs train much better on small inputs) ----
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(passengers)

# ---- Function to build supervised sliding-window sequences ----
# For each time step t: X = values [t-seq_len ... t-1], y = value at t
def create_sequences(data, seq_length=12):
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        x = data[i:(i + seq_length)]
        y = data[i + seq_length]
        xs.append(x)
        ys.append(y)
    return np.array(xs), np.array(ys)

SEQ_LENGTH = 12 # 12-month lookback = one full annual seasonal cycle
X_seq, y_seq = create_sequences(scaled_data, seq_length=SEQ_LENGTH)

# ---- Temporal split: 80% train, 20% test WITHOUT shuffling (time order matters) ----
train_size = int(len(X_seq) * 0.80)
X_train, X_test = X_seq[:train_size], X_seq[train_size:]
y_train, y_test = y_seq[:train_size], y_seq[train_size:]

print(f"Total sequences created: {len(X_seq)}")
print(f"Training sequences: {X_train.shape[0]} (Shape: {X_train.shape})")
```

```
print(f"Testing sequences: {X_test.shape[0]} (Shape: {X_test.shape})")

# ---- DataLoader for mini-batch training (shuffle only the training set) ----
train_loader = DataLoader(
    TensorDataset(torch.tensor(X_train, dtype=torch.float32), torch.tensor(y_train, dtype=torch.float32)),
    batch_size=16,
    shuffle=True
)
```

PyTorch version: 2.14.0+cpu on cpu
Dataset Head:

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

Total sequences created: 132
Training sequences: 105 (Shape: (105, 12, 1))
Testing sequences: 27 (Shape: (27, 12, 1))

1. Model Architectures

Step-by-Step Algorithm (Vanilla RNN)

1. Start with an initial hidden state h_0 (usually zeros).
2. For each time step t : $h_t = \tanh(W_{ih} x_t + b_{ih} + W_{hh} h_{t-1} + b_{hh})$.
3. Keep only the final hidden state h_T and map it to the forecast: $\hat{y} = W_{ho} h_T + b_o$.
4. Train with backpropagation through time (BPTT): gradients flow backwards through every time step.
5. Limitation: repeated multiplication makes gradients vanish/explode on long sequences.

Step-by-Step Algorithm (LSTM)

1. At each step compute the **forget gate** f_t (what to discard from the cell state).
2. Compute the **input gate** i_t and candidate memory $\tilde{c}_t$ (what new information to store).
3. Update the cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (additive "conveyor belt").
4. Compute the **output gate** o_t and hidden state: $h_t = o_t \odot \tanh(c_t)$.
5. Predict from the last hidden state as in the vanilla RNN.
6. Train with BPTT - the additive cell state lets gradients flow across long horizons.

Training Process (shared)

1. Normalize the series and cut it into sliding windows of 12 months (one seasonal cycle).
2. Split chronologically: first 80% for training, last 20% for testing.
3. For every epoch: forward pass through the recurrent layers, MSE loss, BPTT, Adam update.
4. Track the loss per epoch to compare convergence of RNN vs LSTM.

```
class VanillaRNN(nn.Module):
    """Elman RNN: 2 stacked recurrent layers + a linear output layer."""
    def __init__(self, input_size=1, hidden_size=64, num_layers=2):
        super(VanillaRNN, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1) # map final hidden state to a scalar forecast

    def forward(self, x):
```

```

        out, _ = self.rnn(x)                # out: (batch, seq_len, hidden)
        out = self.fc(out[:, -1, :])        # use ONLY the last time step's hidden state
        return out

class LSTMModel(nn.Module):
    """LSTM: same interface, but gated cells preserve long-term information."""
    def __init__(self, input_size=1, hidden_size=64, num_layers=2):
        super(LSTMModel, self).__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, _ = self.lstm(x)
        out = self.fc(out[:, -1, :])
        return out

# Instantiate both models on the selected device so they can be compared fairly
rnn_model = VanillaRNN().to(device)
lstm_model = LSTMModel().to(device)
print("Models instantiated successfully!")

```

Models instantiated successfully!

```

# ---- Generic training function shared by both architectures ----
def train_model(model, loader, epochs=120, lr=0.005):
    criterion = nn.MSELoss()                # regression -> mean squared error
    optimizer = optim.Adam(model.parameters(), lr=lr) # Adam optimizer
    losses = []

    for epoch in range(epochs):
        model.train()
        total_loss = 0.0
        for x_b, y_b in loader:
            x_b, y_b = x_b.to(device), y_b.to(device)

            optimizer.zero_grad()            # clear previous gradients
            pred = model(x_b)                # forward pass through the recurrent net
            loss = criterion(pred, y_b)       # MSE between forecast and truth
            loss.backward()                  # backpropagation through time (BPTT)
            optimizer.step()                 # update weights

            total_loss += loss.item() * len(x_b) # de-average the batch loss

        epoch_loss = total_loss / len(loader.dataset)
        losses.append(epoch_loss)            # track loss per epoch for plotting

    return losses

# ---- Train both architectures with identical hyperparameters ----
print("Training Vanilla RNN...")
rnn_losses = train_model(rnn_model, train_loader, epochs=120, lr=0.005)

print("Training LSTM Model...")
lstm_losses = train_model(lstm_model, train_loader, epochs=120, lr=0.005)
print("Training completed!")

```

```

Training Vanilla RNN...
Training LSTM Model...
Training completed!

```

2. Evaluation & Forecast

Evaluation & Forecasting Process

1. Switch both models to evaluation mode and disable gradient computation.
2. Predict all test windows, then invert the MinMax scaling to recover real passenger counts.
3. Compute RMSE and MAE per model in the original unit.
4. Plot the forecast trajectories against the ground truth to inspect trend/seasonality fit.

```

# ---- Evaluate both models on the test horizon ----
rnn_model.eval()
lstm_model.eval()

with torch.no_grad(): # inference only - no gradients
    x_test_t = torch.tensor(X_test, dtype=torch.float32).to(device)
    pred_rnn_scaled = rnn_model(x_test_t).cpu().numpy() # normalized predictions
    pred_lstm_scaled = lstm_model(x_test_t).cpu().numpy()

# ---- Invert scaling to get real passenger counts ----
y_test_true = scaler.inverse_transform(y_test)
pred_rnn = scaler.inverse_transform(pred_rnn_scaled)
pred_lstm = scaler.inverse_transform(pred_lstm_scaled)

# ---- Compute error metrics in the original unit (passengers) ----
rmse_rnn = root_mean_squared_error(y_test_true, pred_rnn)
mae_rnn = mean_absolute_error(y_test_true, pred_rnn)

rmse_lstm = root_mean_squared_error(y_test_true, pred_lstm)
mae_lstm = mean_absolute_error(y_test_true, pred_lstm)

# ---- Summary table (lower errors are better) ----
eval_df = pd.DataFrame([
    {"Model": "Vanilla RNN", "RMSE (Passengers)": rmse_rnn, "MAE (Passengers)": mae_rnn},
    {"Model": "LSTM Network", "RMSE (Passengers)": rmse_lstm, "MAE (Passengers)": mae_lstm}
]).set_index("Model")

print("Time Series Forecasting Performance Comparison:")
display(eval_df.style.highlight_min(subset=['RMSE (Passengers)', 'MAE (Passengers)'], color='lightgreen'))

# ---- Plot ground truth vs. both forecast trajectories ----
# Map each test target back to its true position on the month axis:
# sequence i predicts month (i + SEQ_LENGTH)
test_indices = range(train_size + SEQ_LENGTH, len(passengers))

plt.figure(figsize=(12, 6))
# Full historical series in the background
plt.plot(range(len(passengers)), passengers, label='Ground Truth Historical', color='black', alpha=0.5,
lw=1.5)

# Test-period ground truth
plt.plot(test_indices, y_test_true, label='Ground Truth (Test Horizon)', color='black', lw=2.5)

# Model forecasts
plt.plot(test_indices, pred_rnn, label=f'Vanilla RNN (RMSE: {rmse_rnn:.1f})', color='crimson', lw=2,
linestyle='--')
plt.plot(test_indices, pred_lstm, label=f'LSTM Network (RMSE: {rmse_lstm:.1f})', color='teal', lw=2.2)

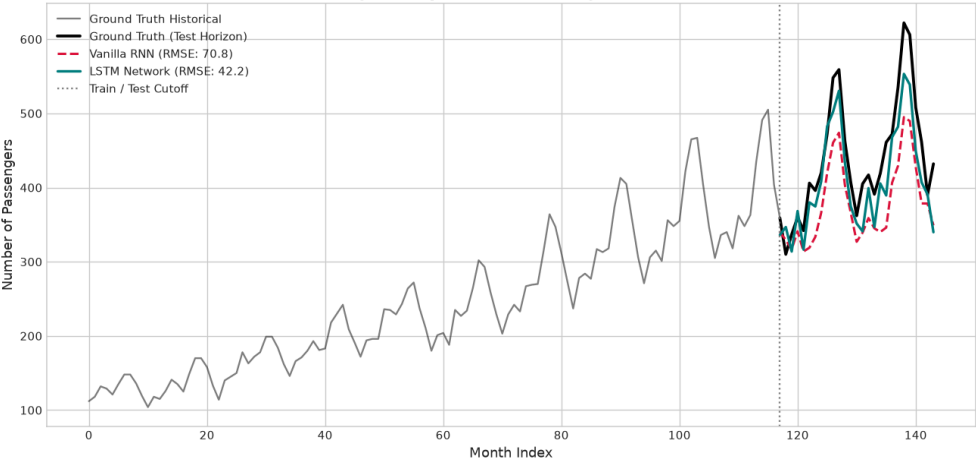
# Vertical line marking where the train period ends and test begins
plt.axvline(x=train_size + SEQ_LENGTH, color='gray', linestyle=':', label='Train / Test Cutoff')
plt.title("Airline Passengers Sequence Forecasting: Vanilla RNN vs LSTM", fontsize=14, fontweight='bold')
plt.xlabel("Month Index", fontsize=12)
plt.ylabel("Number of Passengers", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()

```

Time Series Forecasting Performance Comparison:

	RMSE (Passengers)	MAE (Passengers)
Model		
Vanilla RNN	70.827942	63.158745
LSTM Network	42.171574	34.881374

Airline Passengers Sequence Forecasting: Vanilla RNN vs LSTM



07. Self-Organizing Map (SOM)

Algorithm implemented: Kohonen Self-Organizing Map (via MiniSom).

Dataset: UCI Wine (178 samples, 13 chemical features, 3 cultivars used only for visualization).

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + scaling ----
from sklearn.datasets import load_wine
from sklearn.preprocessing import MinMaxScaler
# ---- MiniSom: lightweight Kohonen Self-Organizing Map implementation ----
from minisom import MiniSom

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("MiniSom and libraries loaded successfully!")

# ---- Load the Wine recognition dataset ----
wine = load_wine()
X_raw = wine.data          # 178 samples x 13 chemical features
y = wine.target            # 3 wine cultivars (only used for visualization - SOM is unsupervised)
feature_names = wine.feature_names
target_names = wine.target_names

print(f"Samples: {X_raw.shape[0]}, Features: {X_raw.shape[1]}, Classes: {target_names.tolist()}")

# ---- MinMax scaling to [0, 1] ----
# SOM weight vectors live in the same range as the inputs, so scaling keeps learning stable.
scaler = MinMaxScaler()
X = scaler.fit_transform(X_raw)
```

```
MiniSom and libraries loaded successfully!
Samples: 178, Features: 13, Classes: ['class_0', 'class_1', 'class_2']
```

Kohonen SOM

Step-by-Step Algorithm (Kohonen Self-Organizing Map)

1. **Initialize** an $M \times N$ grid of neurons, each with a random weight vector w_j of the same dimension as the input.
2. **Sample** a training input x (randomly, with replacement).
3. **Competitive phase:** find the Best Matching Unit (BMU) $c = \operatorname{argmin}_j \|x - w_j\|$.
4. **Cooperative phase:** compute the Gaussian neighborhood kernel $h_{cj} = \exp(-\|r_c - r_j\|^2 / (2\sigma(t)^2))$.
5. **Adaptive phase:** move the BMU and its neighbors toward x : $w_j \leftarrow w_j + \alpha(t) h_{cj} (x - w_j)$.
6. **Decay** the learning rate $\alpha(t)$ and neighborhood radius $\sigma(t)$ after each step.
7. **Repeat** steps 2-6 for many iterations until the map stabilizes.
8. **Output:** a 2D topology-preserving map; each sample is assigned to the neuron (BMU) closest to it.

How the U-Matrix Is Built

1. Train the SOM until the weight vectors organize.

2. For every neuron compute the average Euclidean distance to its immediate grid neighbors.
3. Render these averages as a heatmap: bright ridges separate clusters, dark valleys are cluster interiors.
4. Project each sample onto its BMU using the marker of its true cultivar to check unsupervised separation.

```
# ---- Configure the Kohonen grid ----
grid_x, grid_y = 12, 12 # 12x12 grid = 144 neurons (fewer than the 178 samples)
input_len = X.shape[1] # 13 chemical features per neuron weight vector

som = MiniSom(
    x=grid_x,
    y=grid_y,
    input_len=input_len,
    sigma=1.5, # initial neighborhood radius
    learning_rate=0.5, # initial learning rate
    neighborhood_function='gaussian', # smooth Gaussian neighborhood kernel
    random_seed=42
)

# ---- Initialize weights with PCA for topologically faithful, faster convergence ----
som.pca_weights_init(X)
print("Initial Quantization Error:", som.quantization_error(X))

# ---- Train: 5000 random samples presented to the map ----
som.train_random(data=X, num_iteration=5000, verbose=False)
final_qe = som.quantization_error(X) # average distance from samples to their BMU
final_te = som.topographic_error(X) # fraction of samples whose 2nd BMU is not a grid neighbor

print(f"Training complete after 5,000 iterations!")
print(f" - Final Quantization Error (mean distance to BMU): {final_qe:.4f}")
print(f" - Final Topographic Error (proportion of foldings): {final_te:.4f}")
```

```
Initial Quantization Error: 0.4447391189659704
Training complete after 5,000 iterations!
 - Final Quantization Error (mean distance to BMU): 0.1965
 - Final Topographic Error (proportion of foldings): 0.0393
```

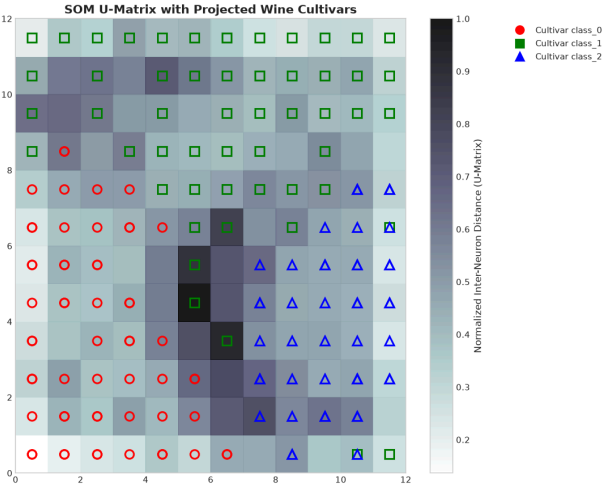
```
# ---- Plot the U-Matrix (unified distance map) ----
# Each cell = average distance between a neuron and its immediate grid neighbors.
# Dark valleys = dense clusters; bright ridges = boundaries between clusters.
plt.figure(figsize=(10, 8))
plt.pcolor(som.distance_map().T, cmap='bone_r', alpha=0.9)
cbar = plt.colorbar()
cbar.set_label("Normalized Inter-Neuron Distance (U-Matrix)", fontsize=11)

# ---- Project every sample onto its Best Matching Unit (BMU) ----
markers = ['o', 's', '^']
colors = ['red', 'green', 'blue']

for idx, x_vec in enumerate(X):
    w = som.winner(x_vec) # grid coordinates of the best matching neuron for this sample
    plt.plot(
        w[0] + 0.5, # +0.5 centers the marker inside the pcolor cell
        w[1] + 0.5,
        markers[y[idx]], # marker shape encodes the true cultivar
        markerfacecolor='None',
        markeredgecolor=colors[y[idx]],
        markersize=10,
        markeredgewidth=2
    )

# Build a Legend for the three cultivars
for c_idx, c_name in enumerate(target_names):
    plt.plot([], [], marker=markers[c_idx], color=colors[c_idx], linestyle='None',
             label=f'Cultivar {c_name}', markersize=9, markeredgewidth=2)

plt.title("SOM U-Matrix with Projected Wine Cultivars", fontsize=14, fontweight='bold')
plt.legend(bbox_to_anchor=(1.25, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



08. Hidden Markov Models (HMM)

Algorithm implemented: Gaussian HMM trained with Baum-Welch (EM) and decoded with the Viterbi algorithm (hmmlearn).

Dataset: Financial market returns & volatility regimes (`data/market_regimes.csv`).

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from hmmlearn.hmm import GaussianHMM

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)

# ---- Load the market returns dataset ----
df_market = pd.read_csv('../data/market_regimes.csv')
print(f"Trading days: {len(df_market)}")
print(df_market.head(3))

returns = df_market['Daily_Return'].values.reshape(-1, 1) # HMM observations
prices = df_market['Close'].values                       # only for visualization
```

```
Trading days: 1000
   Date  Close  Daily_Return  True_Regime
0 2020-01-01  99.41     -0.00587           0
1 2020-01-02  99.68      0.00271           0
2 2020-01-03  99.93      0.00247           0
```

Gaussian HMM

Step-by-Step Algorithm (Gaussian HMM)

1. **Model:** a hidden 1st-order Markov chain Z with transition matrix A , emitting observations $x_t \sim N(\mu_k, \sigma_k^2)$ from the active state.
2. **Learning (Baum-Welch / EM):** iterate until convergence: a. **E-step:** run the forward-backward algorithm to compute posterior state probabilities $\gamma_t(k)$ and $\xi_t(i,j)$. b. **M-step:** re-estimate start probabilities, transition matrix A , and Gaussian parameters (μ_k, σ_k^2) from the posteriors.
3. **Decoding (Viterbi):** dynamic programming finds the single most likely state sequence: $\delta_t(j) = \max_i [\delta_{t-1}(i) A_{ij}] \cdot b_j(x_t)$; $\psi_t(j) = \operatorname{argmax}_i [\dots]$; backtrack from T to 1.
4. **Evaluation:** the forward algorithm gives $P(X | \lambda)$, reported as the model log-likelihood.
5. **Post-processing:** sort states by volatility so regime labels (Bull/Sideways/Bear) are stable and interpretable.

```
# ---- Fit a 3-state Gaussian HMM with the Baum-Welch (EM) algorithm ----
n_components = 3
hmm_model = GaussianHMM(
    n_components=n_components,      # Bull / Sideways / Bear regimes
    covariance_type="full",        # each state has its own full covariance Gaussian emission
    n_iter=200,                   # EM iterations
    random_state=42,
    verbose=False
)

hmm_model.fit(returns)
print("EM Training Converged:", hmm_model.monitor_.converged)
```

```

print(f"Model Log-Likelihood: {hmm_model.score(returns):.2f}")

# ---- Decode the most likely hidden state sequence with the Viterbi algorithm ----
hidden_states = hmm_model.predict(returns)

# ---- Sort regimes by volatility so labels have a consistent economic meaning ----
# Regime 0: Low Volatility (Bull) | Regime 1: Moderate (Sideways) | Regime 2: High (Bear/Crisis)
volatilities = [np.sqrt(hmm_model.covars_[i][0][0]) for i in range(n_components)]
state_order = np.argsort(volatilities) # ascending volatility
state_mapping = {old_state: new_state for new_state, old_state in enumerate(state_order)}

sorted_hidden_states = np.array([state_mapping[s] for s in hidden_states]) # relabel sequence
sorted_means = [hmm_model.means_[old][0] for old in state_order] # reordered means
sorted_stds = [np.sqrt(hmm_model.covars_[old][0][0]) for old in state_order] # reordered std devs

# ---- Reorder the transition matrix to match the sorted regimes ----
sorted_transmat = np.zeros((n_components, n_components))
for i in range(n_components):
    for j in range(n_components):
        sorted_transmat[i, j] = hmm_model.transmat_[state_order[i], state_order[j]]

regime_names = ["Bull (Low Vol)", "Sideways (Med Vol)", "Bear (High Vol)"]

EM Training Converged: True
Model Log-Likelihood: 3018.45

```

```

# ---- Transition probability matrix ----
fig, ax = plt.subplots(figsize=(6, 4.5))
sns.heatmap(sorted_transmat, annot=True, fmt='.3f', cmap='Blues', ax=ax,
            xticklabels=regime_names, yticklabels=regime_names, cbar=False)
ax.set_title("HMM Transition Probability Matrix ($A_{ij}$)", fontweight='bold')
ax.set_xlabel("Transition To Regime")
ax.set_ylabel("Current Regime")
plt.tight_layout()
plt.show()

# ---- Viterbi-decoded regimes over the price series ----
colors = ['forestgreen', 'royalblue', 'crimson']

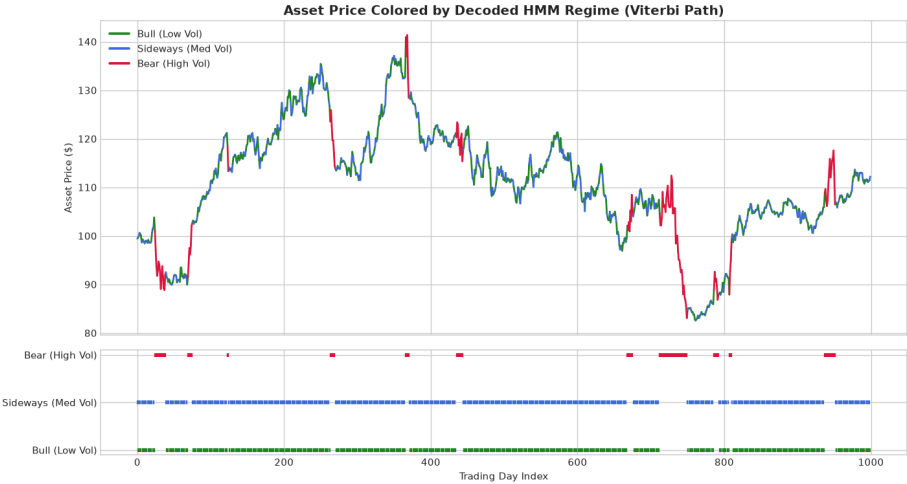
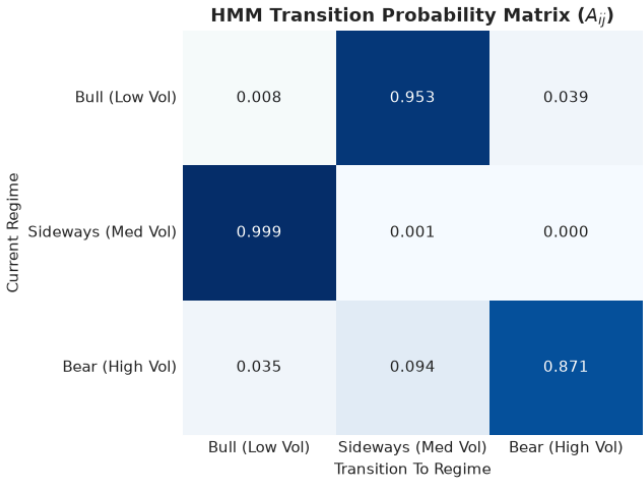
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(13, 7), sharex=True, gridspec_kw={'height_ratios': [3, 1]})

# Top: color every consecutive price segment by its decoded regime
for i in range(len(prices) - 1):
    ax1.plot([i, i + 1], [prices[i], prices[i + 1]], color=colors[sorted_hidden_states[i]], lw=1.8)
for i, name in enumerate(regime_names):
    ax1.plot([], [], color=colors[i], label=name, lw=2.5)
ax1.set_title("Asset Price Colored by Decoded HMM Regime (Viterbi Path)", fontsize=13, fontweight='bold')
ax1.set_ylabel("Asset Price ($)")
ax1.legend(loc='upper left')

# Bottom: the raw sequence of decoded states over time
ax2.scatter(range(len(sorted_hidden_states)), sorted_hidden_states,
            c=[colors[s] for s in sorted_hidden_states], s=15, marker='|')
ax2.set_yticks(range(n_components))
ax2.set_yticklabels(regime_names)
ax2.set_xlabel("Trading Day Index")

plt.tight_layout()
plt.show()

```



09. Support Vector Machines (SVM)

Algorithms implemented: Support Vector Classification (SVC) with linear/poly/RBF kernels and Support Vector Regression (SVR) with an epsilon-insensitive tube.

Datasets: Synthetic two-moons (2D), Breast Cancer (30 features), and a 1D sinusoidal benchmark.

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Datasets + model selection ----
from sklearn.datasets import load_breast_cancer, make_moons
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
# ---- SVM models: classification and regression ----
from sklearn.svm import SVC, SVR
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, r2_score,
root_mean_squared_error

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("SVM modules loaded successfully!")
```

SVM modules loaded successfully!

1. Support Vector Classification (SVC)

Step-by-Step Algorithm (Support Vector Classification)

1. **Formulate** the soft-margin problem: minimize $\frac{1}{2}\|w\|^2 + C \sum \xi_i$ subject to $y_i(w^T\phi(x_i)+b) \geq 1 - \xi_i$, $\xi_i \geq 0$.
2. **Solve** the dual quadratic program with Lagrange multipliers α_i (convex $\rightarrow$ global optimum).
3. **Identify support vectors:** training points with $\alpha_i > 0$; they lie on or inside the margin.
4. **Kernel trick:** replace inner products with $K(x_i, x_j)$ (linear, polynomial, RBF, ...) to model non-linear boundaries.
5. **Predict:** class = $\text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$.
6. **Tune** C and γ by cross-validation to balance margin width and violations.

```
# ---- 2D synthetic data for visualizing margins and kernels ----
# Two interleaving half-moons cannot be separated by a straight line.
X_2d, y_2d = make_moons(n_samples=250, noise=0.20, random_state=42)
scaler_2d = StandardScaler()
X_2d_scaled = scaler_2d.fit_transform(X_2d)

kernels = ['linear', 'poly', 'rbf']
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Coordinate grid used to draw the decision boundary as a contour
x_min, x_max = X_2d_scaled[:, 0].min() - 0.5, X_2d_scaled[:, 0].max() + 0.5
y_min, y_max = X_2d_scaled[:, 1].min() - 0.5, X_2d_scaled[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

for ax, k_name in zip(axes, kernels):
    # Train one SVC per kernel on the same data
    clf = SVC(kernel=k_name, C=1.0, gamma='scale', degree=3, random_state=42)
```

```

clf.fit(X_2d_scaled, y_2d)

# Signed distance to the hyperplane for every grid point
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)

# Filled contour = decision regions; solid line = boundary (Z=0); dashed = margins (Z=±1)
ax.contourf(xx, yy, Z, levels=np.linspace(Z.min(), Z.max(), 20), cmap='coolwarm', alpha=0.3)
ax.contour(xx, yy, Z, levels=[-1.0, 0.0, 1.0], linestyles=['--', '-', '--'], colors=['k', 'k', 'k'],
linewidths=[1.5, 2.5, 1.5])

# Scatter the data points (colored by class)
ax.scatter(X_2d_scaled[:, 0], X_2d_scaled[:, 1], c=y_2d, cmap='coolwarm', edgecolors='k', s=40,
alpha=0.85)

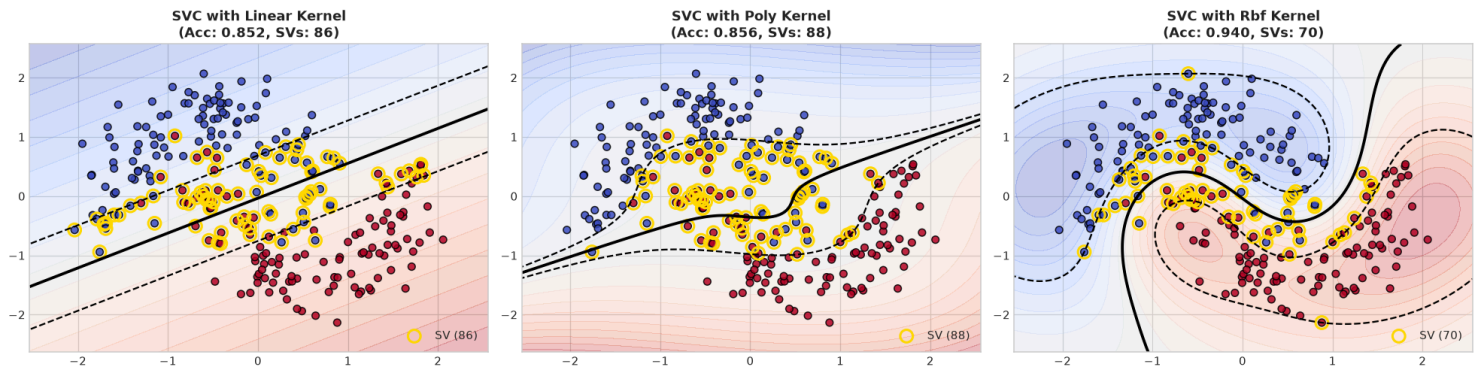
# Highlight the support vectors (the points that define the margin)
sv = clf.support_vectors_
ax.scatter(sv[:, 0], sv[:, 1], s=120, facecolors='none', edgecolors='gold', linewidths=2.0, label=f'SV
({len(sv)})')

ax.set_title(f"SVC with {k_name.capitalize()} Kernel\n(Acc: {clf.score(X_2d_scaled, y_2d):.3f}, SVs:
{len(sv)})", fontweight='bold')
ax.legend(loc='lower right')

plt.suptitle("SVC Decision Boundaries, Margin Tubes, and Support Vectors", fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()

```

SVC Decision Boundaries, Margin Tubes, and Support Vectors



```

# ---- SVC on a real dataset: Breast Cancer (RBF kernel) ----
cancer = load_breast_cancer()
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    cancer.data, cancer.target, test_size=0.25, random_state=42, stratify=cancer.target
)
scaler_c = StandardScaler()
X_train_c_s = scaler_c.fit_transform(X_train_c)
X_test_c_s = scaler_c.transform(X_test_c)

svc_rbf = SVC(kernel='rbf', C=1.0, random_state=42)
svc_rbf.fit(X_train_c_s, y_train_c)
print(f"SVC (RBF) test accuracy on Breast Cancer: {svc_rbf.score(X_test_c_s, y_test_c):.4f}")
print(f"Support vectors used: {len(svc_rbf.support_)} of {len(X_train_c)} training samples")

```

SVC (RBF) test accuracy on Breast Cancer: 0.9790
Support vectors used: 96 of 426 training samples

2. Support Vector Regression (SVR)

Step-by-Step Algorithm (Support Vector Regression)

1. Define an ϵ -insensitive tube: residuals with $|y_i - f(x_i)| \leq \epsilon$ incur zero loss.
2. Minimize $\frac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*)$ subject to the tube constraints.
3. Solve the dual; only points on/outside the tube get $\alpha_i > 0$ and become support vectors.
4. **Predict:** $f(x) = \sum \alpha_i K(x_i, x) + b$ (kernel regression).
5. Visualize the fitted curve, the $\pm\epsilon$ tube, and the support vectors.

```
# ---- 1D non-linear regression demonstration ----
np.random.seed(42)
# Synthetic target: sine + linear trend + Gaussian noise
X_svr = np.sort(np.random.uniform(-3, 3, 120)).reshape(-1, 1)
y_svr = np.sin(X_svr).ravel() + 0.15 * X_svr.ravel() + np.random.normal(0, 0.12, 120)

# epsilon defines the width of the "no penalty" tube around the prediction
epsilon_val = 0.18
svr_model = SVR(kernel='rbf', C=20.0, epsilon=epsilon_val, gamma=0.5)
svr_model.fit(X_svr, y_svr)

# Dense grid for drawing a smooth prediction curve
X_grid = np.linspace(-3.2, 3.2, 300).reshape(-1, 1)
y_grid_pred = svr_model.predict(X_grid)

# Support vectors = training points that lie on/outside the epsilon tube
sv_indices = svr_model.support_
X_sv = X_svr[sv_indices]
y_sv = y_svr[sv_indices]

plt.figure(figsize=(11, 6))
# Training samples
plt.scatter(X_svr, y_svr, color='navy', s=35, alpha=0.75, label='Training Samples')

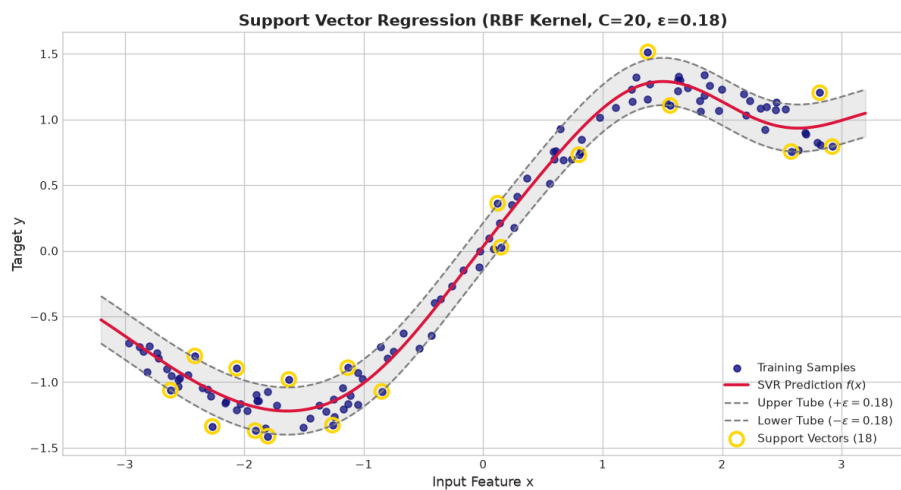
# Regression surface
plt.plot(X_grid, y_grid_pred, color='crimson', lw=2.5, label='SVR Prediction $f(x)$')

# Epsilon-insensitive tube boundaries + shaded tube
plt.plot(X_grid, y_grid_pred + epsilon_val, color='gray', linestyle='--', lw=1.5, label=rf'Upper Tube ($+\epsilon=\{epsilon\_val\}$)')
plt.plot(X_grid, y_grid_pred - epsilon_val, color='gray', linestyle='--', lw=1.5, label=rf'Lower Tube ($-\epsilon=\{epsilon\_val\}$)')
plt.fill_between(X_grid.ravel(), y_grid_pred - epsilon_val, y_grid_pred + epsilon_val, color='gray', alpha=0.15)

# Support vectors are highlighted - only these points influence the fitted curve
plt.scatter(X_sv, y_sv, s=140, facecolors='none', edgecolors='gold', linewidths=2.5, label=f'Support Vectors ({len(X_sv)})')

plt.title(f"Support Vector Regression (RBF Kernel, C=20,  $\epsilon=\{epsilon\_val\}$ )", fontsize=14, fontweight='bold')
plt.xlabel("Input Feature x", fontsize=12)
plt.ylabel("Target y", fontsize=12)
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()

# ---- Quantitative evaluation ----
print(f"SVR R2 Score: {r2_score(y_svr, svr_model.predict(X_svr)):.4f}")
print(f"SVR RMSE: {root_mean_squared_error(y_svr, svr_model.predict(X_svr)):.4f}")
print(f"Support Vectors: {len(X_sv)} out of {len(X_svr)} points ({len(X_sv)/len(X_svr)*100:.1f}%)")
```



SVR R^2 Score: 0.9864

SVR RMSE: 0.1157

Support Vectors: 18 out of 120 points (15.0%)

10. Large Language Models (LLM)

Capabilities implemented: subword tokenization, sentiment inference, autoregressive text generation, and a 1-epoch mini fine-tune of a pre-trained transformer.

Models: DistilBERT (SST-2) and DistilGPT2 via Hugging Face `transformers` .

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- PyTorch: used for the mini fine-tuning demo ----
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
# ---- Hugging Face Transformers: tokenizers, models, pipelines, generation configs ----
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    AutoModelForCausalLM,
    GenerationConfig,
    pipeline
)

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

torch.manual_seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # GPU if available
print(f"PyTorch on {device}")
```

```
D:\ML\Assignment\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update
jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
PyTorch on cpu
```

1. Tokenization

Step-by-Step Algorithm (Transformer Tokenization)

1. Normalize and split the raw text (lowercase, punctuation handling, etc.).
2. Apply subword segmentation (WordPiece/BPE): frequent words stay whole, rare words split into pieces + "##" continuations.
3. Map each subword to its integer vocabulary ID.
4. Add special tokens ([CLS], [SEP]) and pad/truncate to a fixed length.
5. Build the attention mask: 1 for real tokens, 0 for padding.
6. Feed IDs + mask into the transformer's embedding and self-attention layers.

```
# ---- Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based) ----
model_id = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
print(f"Loading Tokenizer: {model_id}...")
tokenizer = AutoTokenizer.from_pretrained(model_id)

sample_text = "Machine learning and deep neural networks are transforming modern science!"
# Tokenize with padding/truncation so the output has a fixed shape (here 20 tokens)
```

```

encoded = tokenizer(
    sample_text,
    padding="max_length",
    max_length=20,
    truncation=True,
    return_tensors="pt"
)

# Convert integer IDs back to readable subword tokens
tokens = tokenizer.convert_ids_to_tokens(encoded['input_ids'][0])
print(f"Original Text: {sample_text}\n")
print(f"Subword Tokens ({len(tokens)}):\n", tokens)
print(f"Input IDs: \n", encoded['input_ids'][0].tolist())
print(f"Attention Mask: \n", encoded['attention_mask'][0].tolist())

```

Loading Tokenizer: distilbert/distilbert-base-uncased-finetuned-sst-2-english...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Original Text: Machine learning and deep neural networks are transforming modern science!

Subword Tokens (20):

['[CLS]', 'machine', 'learning', 'and', 'deep', 'neural', 'networks', 'are', 'transforming', 'modern', 'science', '!', '[SEP]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]']

Input IDs:

[101, 3698, 4083, 1998, 2784, 15756, 6125, 2024, 17903, 2715, 2671, 999, 102, 0, 0, 0, 0, 0, 0]

Attention Mask:

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0]

2. Inference: Sentiment Analysis

Inference Process (Hugging Face Pipeline)

1. Load the pre-trained checkpoint (weights + tokenizer).
2. Tokenize each input sentence exactly as during pre-training.
3. Run a forward pass through the transformer encoder.
4. Apply softmax to the classification head logits to obtain class probabilities.
5. Return the highest-probability label with its confidence score.

```

# ---- Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2) ----
classifier = pipeline("sentiment-analysis", model=model_id, device=-1) # -1 = run on CPU

test_sentences = [
    "The new neural network model demonstrated extraordinary accuracy and remarkable speed.",
    "The dataset was poorly labeled, noisy, and the model completely failed to converge.",
    "The gradient descent algorithm performed reasonably well, meeting standard expectations.",
    "A catastrophic failure occurred during training due to exploding gradient issues."
]

# One forward pass per sentence -> Label + calibrated confidence score
predictions = classifier(test_sentences)

# Collect results in a table for display
results_table = []
for sent, pred in zip(test_sentences, predictions):
    results_table.append({
        "Sentence": sent,
        "Predicted Label": pred['label'],
        "Confidence Score": f"{pred['score']*100:.2f}%"
    })

df_preds = pd.DataFrame(results_table)
display(df_preds)

```

Loading weights: 0%| | 0/104 [00:00<?, ?it/s]

Loading weights: 100%|██████████| 104/104 [00:00<00:00, 1072.71it/s]

	Sentence	Predicted Label	Confidence Score
0	The new neural network model demonstrated extr...	POSITIVE	99.99%
1	The dataset was poorly labeled, noisy, and the...	NEGATIVE	99.98%
2	The gradient descent algorithm performed reaso...	POSITIVE	99.92%
3	A catastrophic failure occurred during trainin...	NEGATIVE	99.96%

3. Text Generation

Step-by-Step Algorithm (Autoregressive Generation)

1. Tokenize the prompt and run the transformer to obtain next-token logits.
2. **Greedy decoding**: pick the single most likely token each step (deterministic, can loop).
3. **Temperature scaling**: divide logits by T before softmax ($T < 1$ sharpens, $T > 1$ flattens).
4. **Top-p (nucleus) sampling**: keep the smallest token set whose cumulative probability $\geq p$, then sample inside it.
5. Append the sampled token to the sequence and repeat until the token budget is reached.
6. Decode the final token IDs back to text.

```
# ---- Load a small causal LM (DistilGPT2) for text generation ----
gen_model_id = "distilbert/distilgpt2"
print(f>Loading Text Generator: {gen_model_id}...")
# clean_up_tokenization_spaces=False keeps BPE spacing intact
generator = pipeline("text-generation", model=gen_model_id, device=-1, clean_up_tokenization_spaces=False)

prompt = "Artificial Intelligence will fundamentally transform"

# 1. Greedy Search: always pick the most likely next token (deterministic)
greedy_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=False)
)[0]['generated_text']

# 2. Temperature + Top-p (Nucleus) Sampling: sample from the most likely tokens up to cumulative p
sampled_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=True, temperature=0.7,
    top_p=0.9)
)[0]['generated_text']

print(f>PROMPT: '{prompt}'\n")
print(f"--- 1. GREEDY SEARCH OUTPUT ---\n{greedy_output.strip()}\n")
print(f"--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---\n{sampled_output.strip()}")
```

Loading Text Generator: distilbert/distilgpt2...

Loading weights: 0%| | 0/76 [00:00<?, ?it/s]

Loading weights: 18%|██ | 14/76 [00:00<00:01, 60.63it/s]

Loading weights: 100%|██████████| 76/76 [00:00<00:00, 297.80it/s]

PROMPT: 'Artificial Intelligence will fundamentally transform'

--- 1. GREEDY SEARCH OUTPUT ---

Artificial Intelligence will fundamentally transform the way we think about the world.

--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---

Artificial Intelligence will fundamentally transform the way people think about the world.

4. Mini Fine-Tuning

Fine-Tuning Process (Sequence Classification)

1. Build a small labeled dataset of domain queries (Technical vs Billing).

2. Tokenize all texts with padding/truncation and wrap them in a PyTorch Dataset/DataLoader.
3. Load the pre-trained transformer with a new randomly-initialized 2-class head.
4. For each epoch: forward pass, cross-entropy loss, backpropagation, AdamW update (small LR).
5. Track the loss curve across epochs.
6. Evaluate on unseen queries: softmax the logits and report predicted class + confidence.

```
# ---- Tiny custom dataset: classify support queries as Technical (0) or Billing (1) ----
train_texts = [
    "My model throws a CUDA out of memory error during backpropagation",
    "How do I fix the gradient vanishing issue in my deep recurrent network?",
    "The API returns a 500 internal server error when requesting predictions",
    "Where can I find the documentation for custom PyTorch loss functions?",
    "Why was my credit card charged twice for the monthly subscription?",
    "I would like to request an invoice for my corporate license",
    "Can you refund my annual subscription fee?",
    "How can I upgrade my billing tier to enterprise?"
]
train_labels = [0, 0, 0, 0, 1, 1, 1, 1] # 0 = Technical, 1 = Billing

# Tokenize the whole dataset once (padded, with attention masks)
encodings = tokenizer(train_texts, padding=True, truncation=True, return_tensors="pt")

class MiniTextDataset(Dataset):
    """Wrap tokenized encodings + labels into a PyTorch Dataset."""
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        # One dictionary per sample: input_ids, attention_mask, Labels
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        return len(self.labels)

dataset = MiniTextDataset(encodings, train_labels)
dataloader = DataLoader(dataset, batch_size=4, shuffle=True)

# ---- Load the pre-trained classifier head with a fresh 2-class head ----
# ignore_mismatched_sizes allows replacing the original SST-2 head with a new one.
ft_model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2,
ignore_mismatched_sizes=True)
ft_model.to(device)
optimizer = torch.optim.AdamW(ft_model.parameters(), lr=2e-5) # small LR for fine-tuning

print("Pre-trained Transformer loaded for fine-tuning!")
```

```
Loading weights:  0%|          | 0/104 [00:00<?, ?it/s]
Loading weights: 17%|██        | 18/104 [00:00<00:00, 173.02it/s]
Loading weights: 100%|██████████| 104/104 [00:00<00:00, 816.45it/s]
Pre-trained Transformer loaded for fine-tuning!
```

```
# ---- Mini fine-tuning loop ----
epochs = 1
ft_model.train()
epoch_losses = []

for epoch in range(epochs):
    running_loss = 0.0
    for batch in dataloader:
        optimizer.zero_grad()

        # Move the batch to the selected device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
```

```

# Forward pass: the model computes the classification loss internally
outputs = ft_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
loss = outputs.loss
loss.backward()      # backpropagate through the whole transformer
optimizer.step()     # update weights

running_loss += loss.item() * len(input_ids)  # de-average the batch loss

avg_loss = running_loss / len(train_texts)
epoch_losses.append(avg_loss)
print(f"Epoch [{epoch+1}/{epochs}] Fine-Tuning Loss: {avg_loss:.4f}")

```

Epoch [1/1] Fine-Tuning Loss: 3.0039

```

# ---- Evaluate the fine-tuned model on unseen domain queries ----
ft_model.eval()
eval_texts = [
    "I need to cancel my subscription payment immediately",      # expected: Billing (1)
    "Can you help me debug a torch dimension mismatch tensor error?" # expected: Technical (0)
]

with torch.no_grad():
    eval_enc = tokenizer(eval_texts, padding=True, truncation=True, return_tensors="pt").to(device)
    eval_outputs = ft_model(**eval_enc)                        # forward pass only
    eval_probs = torch.softmax(eval_outputs.logits, dim=1).cpu().numpy() # logits -> probabilities
    eval_preds = np.argmax(eval_probs, axis=1)                 # predicted class per query

# Human-readable class names
labels_map = {0: "Technical Support", 1: "Billing & Accounts"}
for text, pred, prob in zip(eval_texts, eval_preds, eval_probs):
    print(f"Query:      '{text}'")
    print(f"Prediction: {labels_map[pred]} (Confidence: {prob[pred]*100:.1f}%)\\n")

```

Query: 'I need to cancel my subscription payment immediately'
Prediction: Technical Support (Confidence: 99.9%)

Query: 'Can you help me debug a torch dimension mismatch tensor error?'
Prediction: Technical Support (Confidence: 98.9%)

11. Generalized Regression Neural Network (GRNN)

Algorithm implemented: Specht's GRNN (Nadaraya-Watson kernel regression) from scratch.

Dataset: Noisy 1D multi-modal benchmark function.

```
# ---- Core Libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.base import BaseEstimator, RegressorMixin
from sklearn.model_selection import KFold, train_test_split
from sklearn.metrics import r2_score, root_mean_squared_error

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

GRNN Architecture

Step-by-Step Algorithm (Generalized Regression Neural Network)

1. **Input layer:** receive the d -dimensional query vector x and pass it to every pattern neuron.
2. **Pattern layer:** with N training points, each unit i computes a Gaussian activation $p_i(x) = \exp(-||x - x_i||^2 / (2\sigma^2))$.
3. **Summation layer:** compute two sums over all pattern units:
 - $S(x) = \sum_i y_i \cdot p_i(x)$ (weighted by training targets)
 - $D(x) = \sum_i p_i(x)$ (normalization)
4. **Output layer:** predict the conditional mean $\hat{y}(x) = S(x) / D(x)$ (Nadaraya-Watson estimator).
5. **Training:** one pass - just store the training set; **no** backpropagation or iterative optimization.
6. **Tuning:** only σ must be chosen (small σ = memorizes noise; large σ = over-smooth).

```
class GRNN(BaseEstimator, RegressorMixin):
    """
    Generalized Regression Neural Network (Specht, 1991) - vectorized implementation.

    Prediction is the Nadaraya-Watson kernel regression estimate:
        y_hat(x) = sum_i y_i * exp(-||x - x_i||^2 / (2*sigma^2))
                  / sum_i      exp(-||x - x_i||^2 / (2*sigma^2))

    Training is one-pass: the model simply stores the training set (no gradient descent).
    The only hyperparameter is the smoothing spread sigma.
    """

    def __init__(self, sigma=1.0):
        self.sigma = sigma

    def fit(self, X, y):
        # "Training" = memorizing the data (pattern + summation layers)
        self.X_train_ = np.asarray(X, dtype=np.float64)
        self.y_train_ = np.asarray(y, dtype=np.float64).ravel()
        return self

    def predict(self, X):
        X_test = np.asarray(X, dtype=np.float64)
```

```

# ---- Pattern Layer: squared Euclidean distance between each test and train point ----
# Uses the expansion ||a-b||^2 = ||a||^2 + ||b||^2 - 2 a.b for efficiency.
dist_sq = np.sum(X_test**2, axis=1, keepdims=True) + \
            np.sum(self.X_train**2, axis=1, keepdims=True).T - \
            2 * np.dot(X_test, self.X_train.T)
dist_sq = np.maximum(dist_sq, 0.0) # guard against tiny negative values from rounding

# ---- Pattern Layer activation: Gaussian kernel of each training point ----
kernels = np.exp(-dist_sq / (2.0 * (self.sigma ** 2)))

# ---- Summation Layer: D-sum (denominator) and S-sum (numerator) ----
D = np.sum(kernels, axis=1) # unweighted sum of activations
S = np.dot(kernels, self.y_train_) # activation-weighted sum of targets

# ---- Output Layer: normalized weighted average (prevents 0/0 for isolated queries) ----
D_safe = np.where(D < 1e-12, 1e-12, D)
return S / D_safe

```

```
print("Custom vectorized GRNN class defined!")
```

Custom vectorized GRNN class defined!

Smoothing Parameter σ

The Role of the Smoothing Parameter σ

1. Fit GRNNs with a small, medium and large σ on the same noisy 1D data.
2. Small σ (0.05): each training point dominates its own neighborhood -> wiggly curve, high variance.
3. Large σ (1.50): all points influence every query -> flat curve, high bias.
4. Medium σ (≈ 0.35): smooth interpolation that follows the true underlying function.
5. Compare training R^2 and curve shapes to see the bias-variance trade-off.

Hyperparameter Optimization Process (5-Fold CV)

1. Define a grid of candidate σ values from 0.05 to 1.0.
2. Split the data into 5 fixed folds.
3. For each σ : fit on 4 folds, compute RMSE on the held-out fold, then average the 5 scores.
4. Select the σ with the lowest average CV RMSE.
5. Plot the CV error curve to confirm a clear optimum.

```

# ---- Synthetic 1D regression problem ----
np.random.seed(42)
X_syn = np.sort(np.random.uniform(-3, 3, 100)).reshape(-1, 1)
# True latent function: sin(2x) + cos(0.5x^2); observed targets add Gaussian noise
y_true_curve = lambda x: np.sin(2 * x) + np.cos(0.5 * x**2)
y_syn = y_true_curve(X_syn).ravel() + np.random.normal(0, 0.18, 100)

X_eval = np.linspace(-3.2, 3.2, 300).reshape(-1, 1) # dense grid for plotting

# Under-smoothing, near-optimal, over-smoothing
sigmas = [0.05, 0.35, 1.50]
colors = ['purple', 'crimson', 'teal']

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, sig, col in zip(axes, sigmas, colors):
    # Fit a fresh GRNN for each sigma (training is instant)
    grnn_demo = GRNN(sigma=sig).fit(X_syn, y_syn)
    y_pred_demo = grnn_demo.predict(X_eval)

    ax.scatter(X_syn, y_syn, color='navy', s=35, alpha=0.7, label='Training Data')
    ax.plot(X_eval, y_true_curve(X_eval), 'k--', lw=1.5, label='True Signal')

```

```

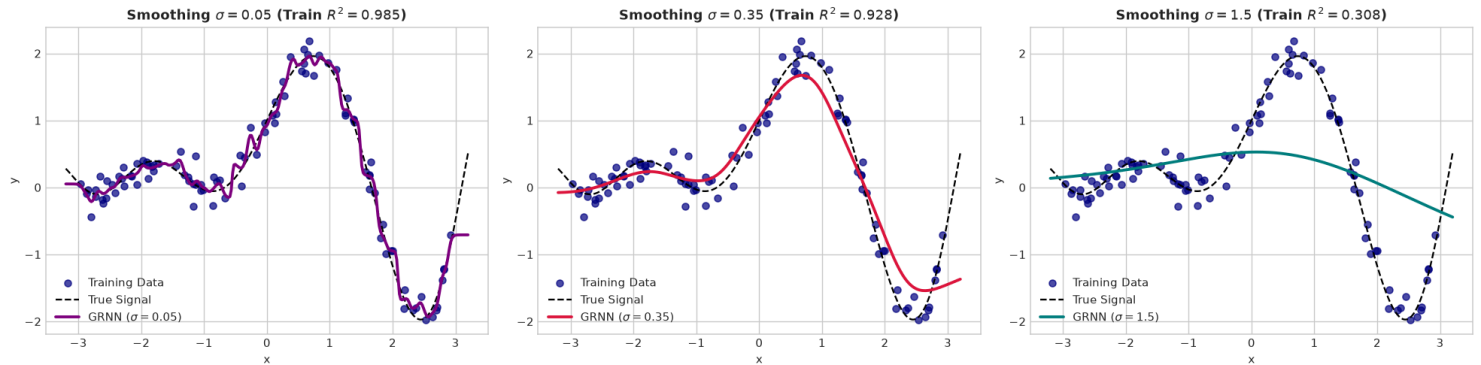
ax.plot(X_eval, y_pred_demo, color=col, lw=2.5, label=rf'GRNN ( $\sigma$ ={sig})')

# Training R^2 shows the bias-variance behavior for each sigma
r2 = r2_score(y_syn, grnn_demo.predict(X_syn))
ax.set_title(rf"Smoothing  $\sigma$  = {sig} (Train  $R^2$  = {r2:.3f})", fontweight='bold')
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend(loc='lower left')

plt.suptitle("GRNN Behavior Under Different Smoothing Spread Values ( $\sigma$ )", fontsize=15, fontweight='bold')
plt.tight_layout()
plt.show()

```

GRNN Behavior Under Different Smoothing Spread Values (σ)



```

# ---- 5-fold CV grid search for the optimal smoothing sigma ----
sigma_candidates = np.linspace(0.05, 1.0, 50)
cv_scores = []
kf = KFold(n_splits=5, shuffle=True, random_state=42) # fixed folds for a fair comparison

for sig in sigma_candidates:
    fold_rmse = []
    for train_idx, val_idx in kf.split(X_syn):
        # Split the 1D data into this fold's train/validation parts
        X_tr, y_tr = X_syn[train_idx], y_syn[train_idx]
        X_va, y_va = X_syn[val_idx], y_syn[val_idx]

        # Fit GRNN on the training fold and score on the validation fold
        model = GRNN(sigma=sig).fit(X_tr, y_tr)
        preds = model.predict(X_va)
        fold_rmse.append(root_mean_squared_error(y_va, preds))

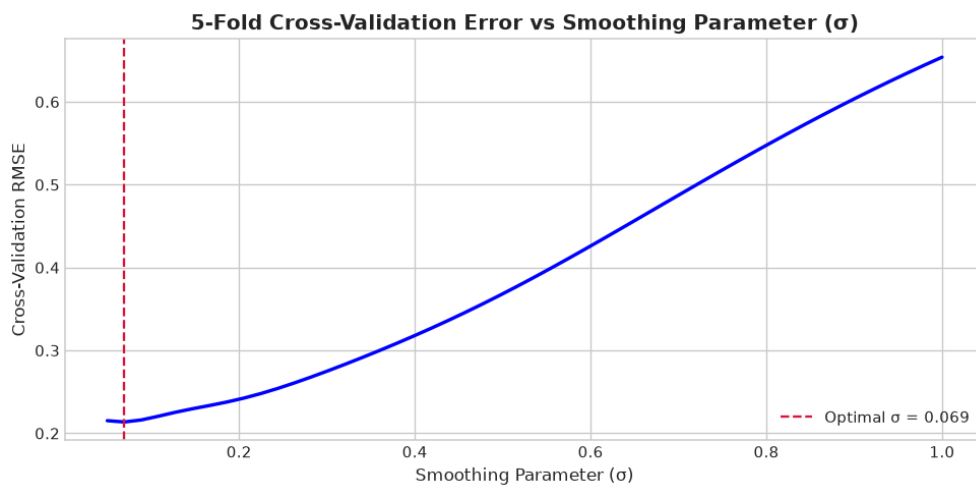
    cv_scores.append(np.mean(fold_rmse)) # average CV error for this sigma

best_idx = np.argmin(cv_scores)
best_sigma = sigma_candidates[best_idx]
print(f"Optimal Smoothing Parameter  $\sigma$ : {best_sigma:.3f} (CV RMSE: {cv_scores[best_idx]:.4f})")

# ---- Plot the CV error curve and mark the optimum ----
plt.figure(figsize=(9, 4.5))
plt.plot(sigma_candidates, cv_scores, 'b-', lw=2.2)
plt.axvline(x=best_sigma, color='crimson', linestyle='--', label=f'Optimal  $\sigma$  = {best_sigma:.3f}')
plt.title("5-Fold Cross-Validation Error vs Smoothing Parameter ( $\sigma$ )", fontsize=13, fontweight='bold')
plt.xlabel("Smoothing Parameter ( $\sigma$ )", fontsize=11)
plt.ylabel("Cross-Validation RMSE", fontsize=11)
plt.legend()
plt.tight_layout()
plt.show()

```

Optimal Smoothing Parameter σ : 0.069 (CV RMSE: 0.2137)



```
# ---- Final model: refit with the CV-selected sigma and evaluate on a held-out split ----
X_tr, X_te, y_tr, y_te = train_test_split(X_syn, y_syn, test_size=0.25, random_state=42)
final_model = GRNN(sigma=best_sigma).fit(X_tr, y_tr)
y_hat = final_model.predict(X_te)

print(f"Final GRNN with CV-selected sigma = {best_sigma:.3f}")
print(f"- Train R²: {r2_score(y_tr, final_model.predict(X_tr)):.4f}")
print(f"- Test R²: {r2_score(y_te, y_hat):.4f}")
print(f"- Test RMSE: {root_mean_squared_error(y_te, y_hat):.4f}")
```

Final GRNN with CV-selected sigma = 0.069

- Train R²: 0.9829
- Test R²: 0.9473
- Test RMSE: 0.1718